

BOOK 12 — SPRING BOOT PRODUCTION MASTERY

Building Real REST APIs (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 12 of 24 **Spring Versions**

Covered: Spring Boot 3.x (Jakarta EE namespace), Java 17/21 baseline **Prerequisites:** Book 09 (JDBC), Book 10 (HTTP/REST/servlets/JSON/auth concepts), Book 11 (Spring Core — IoC/DI/AOP, all directly automated here) **Next Book:** 13_Spring_Data_JPA_Hibernate.md

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: Book 10 లో servlets manual నా రాశాము. Book 11 లో Spring IoC container ఎలా పనిచేస్తుందో నేర్చుకున్నాము. ఈ పుస్తకం లో, Spring Boot ఆ రెండింటినీ ఎలా **production-ready REST APIs** నా combine చేస్తుందో నేర్చుకుంటాము — auto-configuration, embedded server, `@RestController`, validation, global exception handling, logging, Actuator — real backend engineers రోజూ రాసే code.

English: Book 10 taught raw servlets. Book 11 taught how the Spring IoC container works. This book teaches how Spring Boot combines both into production-ready REST APIs — auto-configuration, the embedded server, `@RestController`, validation, global exception handling, logging, Actuator — the code real backend engineers write every day.

Learning Objectives

1. Understand Spring Boot's auto-configuration and how it builds on Spring Core (Book 11).
 2. Understand standard Spring Boot project structure and starter dependencies.
 3. Build REST controllers with the full `@RequestMapping` family.
 4. Handle requests/responses correctly with DTOs, `@RequestBody`, `@PathVariable`, `@RequestParam`.
 5. Apply Bean Validation (`@Valid`) for input validation.
 6. Build global exception handling with `@RestControllerAdvice`.
 7. Structure Service/Repository layers correctly (preview of full JPA in Book 13).
 8. Manage configuration via `application.properties / .yaml`, `@ConfigurationProperties`, and profiles.
 9. Apply structured logging.
 10. Use Spring Boot Actuator for production observability.
 11. Understand the testing approach for Spring Boot apps (full depth in Book 15).
 12. Build, package, and understand basic deployment readiness.
-

Table of Contents

Ch.	Title
1	Spring Boot Architecture & Auto-Configuration
2	Project Structure & Starters
3	Building REST Controllers
4	Request/Response Handling: DTOs & Parameter Binding
5	Validation with Bean Validation
6	Global Exception Handling
7	Service & Repository Layers
8	Configuration Management & Profiles
9	Logging
10	Spring Boot Actuator
11	Testing Spring Boot Applications (Preview)
12	Building a Complete CRUD REST API
13	Production Readiness: Packaging & Deployment Basics
14	Mini Project — Production-Style REST API
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — Spring Boot Architecture & Auto-Configuration

Telugu Explanation

Spring Boot Spring Framework మీద ఒక **opinionated layer** — manual configuration (Book 11 లో మనం `@Configuration` / `@ComponentScan` manual గా రాశాము) ని **auto-configuration** ద్వారా eliminate చేస్తుంది. `@SpringBootApplication` ఒక్క annotation, మూడు annotations కలిపి: `@Configuration` + `@ComponentScan` + `@EnableAutoConfiguration`. Embedded server (Tomcat, Book 10 Ch.2) కూడా application JAR లోపలే ఉంటుంది — separate deployment అవసరం లేదు.

Professional English Explanation

Spring Boot is an **opinionated layer** on top of Spring Framework — eliminating the manual configuration Book 11 required (`@Configuration` / `@ComponentScan` written by hand) via **auto-configuration**. `@SpringBootApplication` is a single annotation combining three: `@Configuration` + `@ComponentScan` + `@EnableAutoConfiguration`. An embedded server (Tomcat, Book 10 Ch.2) ships inside the application JAR itself — no separate application server deployment required.

Java Code

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;

@SpringBootApplication // = @Configuration + @ComponentScan
+ @EnableAutoConfiguration
public class TaskManagerApplication {
    public static void main(String[] args) {
        SpringApplication.run(TaskManagerApplication.class, args); // starts embedded
        Tomcat + Spring context together
    }
}

@RestController // Book 10 Ch.5's REST principles,
// Book 11 Ch.4's stereotype
class HealthController {
    @GetMapping("/health")
    Map<String, String> health() {
        return Map.of("status", "UP"); // auto-serialized to JSON via
        Jackson (Book 10, Ch.6)
    }
}
```

Diagram — What `@SpringBootApplication` and `run()` Actually Do

```
SpringApplication.run(TaskManagerApplication.class, args)
|
v
1. Create an ApplicationContext (Book 11, Ch.2) - specifically a web-aware one
|
v
2. @ComponentScan discovers @Component/@Service/@Repository/@RestController (Book 11, Ch.4)
|
v
3. @EnableAutoConfiguration inspects the CLASSPATH and configures beans conditionally:
   - spring-boot-starter-web on classpath? -> auto-configure embedded Tomcat +
   DispatcherServlet (Book 10, Ch.2)
   - a JDBC driver on classpath?           -> auto-configure a DataSource (Book 09, Ch.7's
   connection pool)
   - Jackson on classpath?                 -> auto-configure JSON message converters (Book
   10, Ch.6)
|
v
4. Embedded Tomcat starts, DispatcherServlet registered, application ready to serve requests
```

Internal Working

- `@EnableAutoConfiguration` works via **conditional bean registration** — hundreds of pre-written `@Configuration` classes (bundled inside `spring-boot-autoconfigure`) each carry conditions like `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty` — a `DataSource` auto-configuration only activates *if* a JDBC driver is actually on the classpath, *and* only if you haven't already defined your own `DataSource` bean (letting your explicit configuration always take precedence over the automatic default).
- This is **exactly** Book 11's `@Configuration` / `@Bean` / `@Profile` mechanics, just pre-written by the Spring team and conditionally activated based on what's on your classpath and in your configuration — Spring Boot doesn't introduce new IoC/DI concepts, it automates the *application* of the ones Book 11 already taught.
- The embedded server model (Tomcat bundled inside the runnable JAR, `java -jar app.jar` starts everything) is a deliberate architectural shift from older "deploy a WAR file to an external Tomcat installation" models (Book 10's raw servlet deployment) — this is precisely what makes Spring Boot applications trivially containerizable (Book 16/Docker) and consistent across environments.

Real-World Example

Telugu: `spring-boot-starter-web` dependency ఒక్కటే add చేస్తే, embedded Tomcat, Jackson, validation (Ch.5), అన్నీ auto-configure అవుతాయి — Book 10 లో మనం manual గా చేసిన servlet registration, JSON serialization setup, అన్నీ ఇక్కడ zero-configuration గా జరుగుతాయి. English: Adding just the `spring-boot-starter-web` dependency auto-configures the embedded Tomcat, Jackson, and validation (Ch.5) — everything Book 10 required manual servlet registration and JSON setup for happens here with zero explicit configuration, which is exactly Spring Boot's core value proposition.

Interview Answer

"Spring Boot is an opinionated layer over Spring Framework, eliminating manual configuration via auto-configuration — conditionally-activated pre-written `@Configuration` classes that inspect the classpath and existing bean definitions to wire up sensible defaults (embedded server, DataSource, JSON converters) automatically, always yielding to explicit configuration you provide.

`@SpringBootApplication` combines `@Configuration`, `@ComponentScan`, and `@EnableAutoConfiguration` into one annotation. This is fundamentally Book 11's IoC/DI mechanics, automated at scale."

Cross Questions

- Q: Does Spring Boot introduce a fundamentally different dependency injection model than Spring Core? → A: No — it's the same `ApplicationContext` /bean mechanics from Book 11; Spring Boot's value is auto-configuration and convention-over-configuration on top of that same foundation.
- Q: How does auto-configuration know NOT to override a `DataSource` bean you define yourself? → A: Via `@ConditionalOnMissingBean` — the auto-configuration only activates if no matching bean already exists in the context, letting explicit user configuration always take precedence.
- Q: Why does bundling an embedded server matter for deployment? → A: It makes the application a single, self-contained, runnable artifact (`java -jar app.jar`) rather than requiring a separately-installed and -configured application server — directly enabling consistent, simple containerized deployment (Book 16).

Tricky Questions

- Q: If both `spring-boot-starter-web` (needs Tomcat) and a different embedded server starter are on the classpath, what happens? → A: Auto-configuration conditions are also designed to detect and avoid such conflicts/ambiguity in the common case, but mixing multiple embedded-server starters is generally a misconfiguration to avoid deliberately — Spring Boot expects one clear embedded-server choice per application.
- Q: Can you completely disable a specific piece of auto-configuration you don't want? → A: Yes — via `@SpringBootApplication(exclude = SomeAutoConfiguration.class)` or the equivalent `spring.autoconfigure.exclude` property, useful when you need to fully hand-roll a specific piece of configuration Spring Boot would otherwise provide automatically.

Coding Exercise

L1: Create a minimal Spring Boot application with a single `/health` endpoint and run it. **L2:** Add `spring-boot-starter-web` and observe (via startup logs) which auto-configurations activate. **L3:** Define your own `DataSource` bean and confirm (via logs/behavior) that Spring Boot's auto-configured one yields to it. **L4 (Interview):** Explain what `@SpringBootApplication` expands to and what each part does. **L5 (Senior):** Explain how `@ConditionalOnClass` / `@ConditionalOnMissingBean` work together to make auto-configuration safe and overridable. **L6 (Mastery):** Explain, from memory, why Spring Boot's embedded-server model is a deliberate architectural shift from older WAR-deployment models, and its containerization benefit.

CHAPTER 2 — Project Structure & Starters

Telugu Explanation

Standard Spring Boot project structure: `src/main/java` (application code), `src/main/resources` (`application.properties` / `.yml`, static resources), `src/test/java` (tests, Ch.11). **Starters** (`spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `spring-boot-starter-security`) curated dependency bundles — compatible versions ఒకే చోట, version conflicts avoid చేయడానికి.

Professional English Explanation

Standard Spring Boot project structure: `src/main/java` (application code), `src/main/resources` (`application.properties` / `.yml`, static resources), `src/test/java` (tests, Ch.11). **Starters** (`spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `spring-boot-starter-security`) are curated dependency bundles — mutually compatible versions grouped together, avoiding version-conflict headaches.

Project Structure Diagram

```
my-task-manager/
├── pom.xml (or build.gradle)           -- dependencies, including starters
├── src/
│   ├── main/
│   │   ├── java/com/example/taskmanager/
│   │   │   ├── TaskManagerApplication.java -- @SpringBootApplication entry point
│   │   │   ├── controller/                 -- Ch.3 - @RestController classes
│   │   │   ├── service/                   -- Book 11, Book 10 Ch.9 - @Service classes
│   │   │   ├── repository/               -- Book 09/13 - @Repository interfaces
│   │   │   ├── dto/                      -- Ch.4 - request/response DTOs
│   │   │   ├── entity/                   -- Book 13 - JPA entities
│   │   │   ├── exception/                -- Ch.6 - custom exceptions + handler
│   │   │   └── config/                   -- Ch.8 - @Configuration classes
│   │   └── resources/
│   │       ├── application.properties (or .yml) -- Ch.8 configuration
│   │       └── static/, templates/        -- (rarely used for pure REST APIs)
│   └── test/
│       ├── java/com/example/taskmanager/ -- Ch.11 - @SpringBootTest, unit
├── tests
├── target/ (or build/)                -- compiled output, packaged JAR
└── (Ch.13)
```

pom.xml Starters Example

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>           <!-- REST APIs: Tomcat +
Jackson + Spring MVC -->
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>       <!-- Book 13: Hibernate +
Spring Data JPA -->
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>    <!-- Ch.5: Bean
Validation -->
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>       <!-- Ch.10: production
observability -->
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>           <!-- Ch.11/Book 15:
JUnit, Mockito, MockMvc -->
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>                                   <!-- Book 09's in-
memory DB, for dev/test -->
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

Internal Working

- Each **starter** is itself just a Maven/Gradle artifact with **no code of its own** — it exists purely to transitively pull in a curated, version-aligned set of real dependencies (`spring-boot-starter-web` pulls in `spring-webmvc`, `spring-boot-starter-tomcat`, `jackson-databind`, etc.) — this solves the classic "dependency hell" problem of manually picking compatible versions of a dozen interrelated libraries, since Spring Boot's **Bill of Materials (BOM)** guarantees every starter's transitive dependencies are tested to work together.
- The `com.example.taskmanager` sub-package structure (`controller`, `service`, `repository`, `dto`, `entity`) is a **convention**, not a hard requirement — but it directly mirrors Book 10, Ch.9's layered architecture, and Spring Boot's default component scanning (from the `@SpringBootApplication` class's package downward, Book 11 Ch.4) relies on all your beans living somewhere under that root package.
- `src/test/java` mirroring `src/main/java`'s package structure is a Maven/Gradle build-tool convention (not Spring-specific) that both frameworks use to correctly separate test code from production code while keeping it easy to find the test(s) for a given class.

Real-World Example

Telugu: Real projects "starter" dependency ఒక్కటి add చేస్తే (ఉదా. `spring-boot-starter-data-jpa`), దాని కింద ఉన్న 5-10 individual libraries ని వేటినీ version-match చేయాల్సిన అవసరం లేదు — Spring Boot BOM ఇది guarantee చేస్తుంది, real production dependency-conflict incidents తగ్గిస్తుంది. English: Adding a single starter dependency (say, `spring-boot-starter-data-jpa`) in real projects means never manually version-matching the 5-10 individual libraries underneath it — Spring Boot's BOM guarantees this, meaningfully reducing real production dependency-conflict incidents that plagued pre-Boot Spring projects.

Interview Answer

"Standard Spring Boot structure follows `src/main/java` (application code, conventionally organized into controller/service/repository/dto/entity packages mirroring layered architecture), `src/main/resources` (configuration), `src/test/java` (tests). Starters are curated, version-aligned dependency bundles with no code of their own — they transitively pull in real dependencies whose compatible versions are guaranteed by Spring Boot's BOM, solving the classic dependency-version-conflict problem."

Cross Questions

- Q: Does a starter dependency contain actual application code? → A: No — it's purely a dependency-aggregation artifact, pulling in the real libraries needed (e.g., `spring-boot-starter-web` pulls in Tomcat, Jackson, Spring MVC).
- Q: Why does the sub-package structure (`controller / service / repository`) matter for component scanning? → A: `@ComponentScan` (via `@SpringBootApplication`) scans from the main application class's package downward by default — all your beans need to live somewhere under that root package to be automatically discovered.
- Q: What problem does Spring Boot's BOM solve? → A: The classic "dependency hell" of manually selecting mutually-compatible versions across many interrelated libraries — the BOM guarantees every starter's transitive dependency versions are tested together.

Tricky Questions

- Q: If you place a `@RestController` class OUTSIDE the main application class's package (and its sub-packages), will it be discovered automatically? → A: No — by default, component scanning only covers the main class's package and below; classes elsewhere need either an explicit `@ComponentScan(basePackages=...)` override or must be moved into the scanned package tree.
- Q: Can two starters ever pull in genuinely conflicting transitive dependency versions? → A: In principle, rarely, if you deliberately override a specific dependency's version yourself, bypassing the BOM's guarantee — but for the standard, un-overridden case, this is exactly what the BOM is designed to prevent.

Coding Exercise

L1: Set up a new Spring Boot project with the `web`, `data-jpa`, `validation`, and `h2` starters, and verify it starts successfully. **L2:** Organize a small application into

`controller / service / repository / dto` packages under the main application package. **L3:** Deliberately place a `@RestController` outside the scanned package tree and observe it's not picked up; fix it. **L4 (Interview):** Explain what a Spring Boot starter is and what problem it solves. **L5 (Senior):** Review a project's `pom.xml / build.gradle` with many individually-pinned library versions instead of starters — explain the risk and propose migrating to starters. **L6 (Mastery):** Explain, from memory, why component scanning's default package-scoping matters for where you can place your classes.

CHAPTER 3 — Building REST Controllers

Telugu Explanation

`@RestController` (`@Controller` + `@ResponseBody` కలిపి — response body direct నా JSON నా serialize అవుతుంది, Book 10's manual `PrintWriter` /JSON string building అవసరం లేదు) REST endpoints define చేయడానికి వాడతారు. `@RequestMapping` family: `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping` — Book 10, Ch.5's REST method conventions కి direct annotation mapping.

Professional English Explanation

`@RestController` (`@Controller` + `@ResponseBody` combined — the response body is automatically serialized to JSON, eliminating Book 10's manual `PrintWriter` /JSON string building) defines REST endpoints. The `@RequestMapping` family — `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping` — directly maps onto Book 10, Ch.5's REST method conventions.

Java Code

```
import org.springframework.web.bind.annotation.*;
import org.springframework.http.*;
import java.util.*;

record Task(String id, String title, boolean completed) {}

@RestController
@RequestMapping("/api/tasks") // shared base path for all methods
below
class TaskController {

    private final Map<String, Task> tasks = new LinkedHashMap<>(Map.of(
        "1", new Task("1", "Buy groceries", false),
        "2", new Task("2", "Finish report", true)
    ));

    @GetMapping // GET /api/tasks
    List<Task> getAllTasks() {
        return List.copyOf(tasks.values()); // Book 05, Ch.12 - defensive
    }

    @GetMapping("/{id}") // GET /api/tasks/{id}
    ResponseEntity<Task> getTaskById(@PathVariable String id) {
        Task task = tasks.get(id);
        if (task == null) return ResponseEntity.notFound().build(); // 404 (Book 10,
Ch.1)
        return ResponseEntity.ok(task); // 200
    }

    @PostMapping // POST /api/tasks
    ResponseEntity<Task> createTask(@RequestBody Task newTask) { // JSON body -> Task,
auto-deserialized (Book 10, Ch.6)
        Task created = new Task(UUID.randomUUID().toString(), newTask.title(), false);
        tasks.put(created.id(), created);
        return ResponseEntity.status(HttpStatus.CREATED).body(created); // 201
    }

    @PatchMapping("/{id}") // PATCH /api/tasks/{id}
    ResponseEntity<Task> markCompleted(@PathVariable String id) {
        Task existing = tasks.get(id);
        if (existing == null) return ResponseEntity.notFound().build();
        Task updated = new Task(existing.id(), existing.title(), true);
        tasks.put(id, updated);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}") // DELETE
/api/tasks/{id}
    ResponseEntity<Void> deleteTask(@PathVariable String id) {
        if (tasks.remove(id) == null) return ResponseEntity.notFound().build();
        return ResponseEntity.noContent().build(); // 204 (Book 10,
Ch.1)
    }

    @GetMapping(params = "completed") // GET
/api/tasks?completed=true
    List<Task> getByCompletionStatus(@RequestParam boolean completed) {
```

```

        return tasks.values().stream().filter(t -> t.completed() == completed).toList();    //
Book 07, Ch.6-7
    }
}

```

Sample Requests/Responses

```

GET /api/tasks/1
-> 200 OK
{"id":"1","title":"Buy groceries","completed":false}

GET /api/tasks/999
-> 404 Not Found (empty body)

POST /api/tasks
Body: {"title":"Learn Spring Boot"}
-> 201 Created
{"id":"a3f...", "title":"Learn Spring Boot", "completed":false}

PATCH /api/tasks/1
-> 200 OK
{"id":"1","title":"Buy groceries","completed":true}

DELETE /api/tasks/1
-> 204 No Content (empty body)

```

Internal Working

- `@PathVariable` extracts a value from the URL path (`{id}` in `/api/tasks/{id}`), while `@RequestParam` extracts a query string parameter (`?completed=true`) — both perform automatic type conversion (string $\rightarrow$ boolean, int, etc.) and, for required-by-default parameters, automatically produce a `400 Bad Request` if a required one is missing or has the wrong type, saving the manual `req.getParameter()` + parsing + validation from Book 10, Ch.2.
- `ResponseEntity<T>` gives full control over status code, headers, and body together — this is the idiomatic way to return different status codes from the same method based on business logic (404 for not-found, 200 for found), as opposed to simply returning `T` directly (which always implies `200 OK` unless a `@ResponseStatus` or exception handling, Ch.6, intervenes).
- Behind every one of these annotated methods, Spring's `DispatcherServlet` (Book 10, Ch.2's "central servlet") is doing the actual HTTP-level work — inspecting the incoming request's method and path, matching it against the registered `@RequestMapping` patterns, invoking the matched controller method, and converting its return value to the HTTP response body via Jackson (Book 10, Ch.6) — this chapter's annotations are the declarative interface to exactly that underlying servlet machinery.

Real-World Example

Telugu: ఈ `TaskController` ఖచ్చితంగా Book 10, Ch.10's `TaskManagerServlet` (manual, raw servlet నా రాసినది) కి **అదే functionality**, కానీ boilerplate దాదాపు 90% తగ్గించి — annotation-driven routing, automatic JSON serialization, correct status codes అన్నీ Spring Boot handle చేస్తుంది. English: This `TaskController` provides exactly the same functionality as Book 10, Ch.10's manually-written raw

servlet `TaskManagerServlet` — but with roughly 90% less boilerplate, since annotation-driven routing, automatic JSON serialization, and status-code handling are all managed by Spring Boot instead of hand-written.

Interview Answer

"`@RestController` combines `@Controller` and `@ResponseBody`, automatically serializing return values to JSON. The `@RequestMapping` family (`@GetMapping` / `@PostMapping` / etc.) maps HTTP methods to controller methods, following REST conventions from Book 10. `@PathVariable` extracts URL path segments, `@RequestParam` extracts query parameters, both with automatic type conversion and validation. `ResponseEntity<T>` gives full control over status code, headers, and body for cases needing different responses per business outcome."

Cross Questions

- Q: What's the difference between `@PathVariable` and `@RequestParam`? → A: `@PathVariable` extracts a segment from the URL path itself (`/tasks/{id}`); `@RequestParam` extracts a query string parameter (`?key=value`) — different parts of the URL, used for different purposes (identifying a specific resource vs filtering/options).
- Q: Why use `ResponseEntity<T>` instead of just returning `T` directly? → A: `ResponseEntity` gives explicit control over the HTTP status code and headers per response, essential when the same method needs to return different statuses based on business logic (e.g., 200 if found, 404 if not).
- Q: What does `@RestController` save you from writing manually, compared to Book 10's raw servlets? → A: Manual request-method dispatching (`doGet` / `doPost`), manual `PrintWriter`-based JSON string construction, manual `req.getParameter()` parsing/validation — all automated via annotations and Jackson integration.

Tricky Questions

- Q: If a `@RequestParam` is not marked optional and the client omits it, what happens by default? → A: Spring returns `400 Bad Request` automatically, since required parameters are the default; making a parameter genuinely optional requires `@RequestParam(required = false)` (or an `Optional<T>` parameter type, Book 07 Ch.9).
- Q: Does returning `null` from a `@GetMapping` method that returns `T` directly (not `ResponseEntity<T>`) produce a 404? → A: No — it produces a `200 OK` with an empty/null body by default, which is why `ResponseEntity` (explicitly building a 404 response) is the correct idiom when "not found" needs to be communicated via the status code, not just an empty body.

Coding Exercise

L1: Build a full CRUD controller for a different resource (e.g., `Product`) using all 5 `@RequestMapping` variants. **L2:** Add a query-parameter-based filtering endpoint using `@RequestParam`. **L3:** Reproduce the "returning null from a T-returning method gives 200, not 404" behavior, then fix it using `ResponseEntity`. **L4 (Interview):** Explain what `@RestController` saves you from writing manually, referencing Book 10's raw servlets. **L5 (Senior):** Design a `TaskController` API supporting pagination via query parameters (`?page=0&size=20`), explaining the parameter binding. **L6**

(Mastery): Explain, from memory, the request-processing path from an incoming HTTP request through DispatcherServlet to a controller method and back.

CHAPTER 4 — Request/Response Handling: DTOs & Parameter Binding

Telugu Explanation

DTO (Data Transfer Object) అనేది API request/response shape ని represent చేసే class — internal domain/entity objects (Book 13) నుండి **deliberately వేరు గా** ఉంచుతారు, API contract internal implementation details నుండి decouple చేయడానికి. `@RequestBody` JSON request body ని Java object గా bind చేస్తుంది; response గా return అయిన object automatic గా JSON కి serialize అవుతుంది.

Professional English Explanation

A DTO (Data Transfer Object) represents the API request/response shape — deliberately kept **separate** from internal domain/entity objects (Book 13), decoupling the API contract from internal implementation details. `@RequestBody` binds the JSON request body to a Java object; a returned object is automatically serialized to JSON.

Java Code

```
import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
import com.fasterxml.jackson.annotation.JsonProperty;
import java.time.LocalDateTime;

// INTERNAL domain concept (would be a JPA @Entity in Book 13) - has fields the API should
// NEVER expose
class TaskEntity {
    String id;
    String title;
    boolean completed;
    LocalDateTime createdAt;
    String createdByInternalSystemId; // internal-only - must NEVER leak into
    the API response
}

// REQUEST DTO - only what the CLIENT is allowed to send
record CreateTaskRequest(@JsonProperty("title") String title) {} // client can ONLY set
title - nothing else

// RESPONSE DTO - only what the CLIENT is allowed to see
record TaskResponse(String id, String title, boolean completed, LocalDateTime createdAt) {
    static TaskResponse fromEntity(TaskEntity entity) { // mapping: entity ->
    DTO, ONE place (Book 09, Ch.10)
        return new TaskResponse(entity.id, entity.title, entity.completed, entity.createdAt);
        // NOTE: createdByInternalSystemId is DELIBERATELY excluded here - never reaches the
    client
    }
}

@RestController
@RequestMapping("/api/v2/tasks")
class TaskDtoController {
    private final java.util.Map<String, TaskEntity> store = new java.util.LinkedHashMap<>();

    @PostMapping
    ResponseEntity<TaskResponse> create(@RequestBody CreateTaskRequest request) {
        TaskEntity entity = new TaskEntity();
        entity.id = java.util.UUID.randomUUID().toString();
        entity.title = request.title(); // ONLY title comes
        from the client
        entity.completed = false; // server
        controls this - client CANNOT set it
        entity.createdAt = LocalDateTime.now();
        entity.createdByInternalSystemId = "SYS-INTERNAL-42"; // internal-
        only, never from client input

        store.put(entity.id, entity);
        return ResponseEntity.status(201).body(TaskResponse.fromEntity(entity));
    }

    @GetMapping("/{id}")
    ResponseEntity<TaskResponse> get(@PathVariable String id) {
        TaskEntity entity = store.get(id);
        if (entity == null) return ResponseEntity.notFound().build();
        return ResponseEntity.ok(TaskResponse.fromEntity(entity)); // mapped -
        internal field never leaks
    }
}
```

```
}  
}
```

Sample Interaction

```
POST /api/v2/tasks  
Body: {"title": "Review PR", "completed": true, "createdByInternalSystemId": "HACKED"}  
(client tries to set fields it shouldn't be able to - CreateTaskRequest simply DOESN'T  
HAVE those fields)  
-> 201 Created  
{ "id": "alb2...", "title": "Review PR", "completed": false, "createdAt": "2026-08-  
28T10:15:00"}  
(completed correctly forced to false by the server - the client's attempted override was  
IGNORED, since  
CreateTaskRequest's record only declares a 'title' component - Jackson simply has nowhere  
to bind  
"completed" or "createdByInternalSystemId" from the request into)
```

Internal Working

- Using a dedicated `CreateTaskRequest` DTO (rather than binding directly to `TaskEntity`) is a genuine **security and correctness control**, not just a style preference — this is the direct, practical prevention of a class of bug/vulnerability called **mass assignment**, where a client could otherwise set fields they shouldn't control (like `completed`, or worse, an `isAdmin`-style flag on a different domain) simply by including extra JSON fields in their request, if the API naively bound the request body directly onto an internal entity with more fields than intended.
- `TaskResponse.fromEntity()` centralizes entity-to-DTO mapping in exactly **one place** — directly mirroring Book 09, Ch.10's DAO `mapRow()` pattern — ensuring the internal-only `createdByInternalSystemId` field is consistently, correctly excluded everywhere the entity is converted to an API response, rather than relying on remembering to exclude it manually at every individual endpoint.
- This layered DTO approach connects directly to Book 10, Ch.6's `@JsonIgnore` discussion — DTOs are actually the **preferred, stronger** alternative to relying on `@JsonIgnore` scattered across a shared entity class: a dedicated response DTO structurally *cannot* leak a field it was never given, rather than depending on remembering to annotate every sensitive field correctly on a class also used for other purposes.

Real-World Example

Telugu: Real production APIs `Entity` classes ఎప్పుడూ నేరుగా REST response గా return చేయవు — ఎప్పుడూ DTO layer వాడతారు, mass assignment vulnerabilities నివారించడానికి మరియు API contract ని internal DB schema నుండి స్వతంత్రంగా evolve చేయడానికి (DB column పేరు మారినా, API response shape stable గా ఉండొచ్చు). English: Real production APIs never return `Entity` classes directly as REST responses — a DTO layer is always used, both to prevent mass-assignment vulnerabilities and to let the API contract evolve independently of the internal DB schema (a database column rename doesn't have to break the public API response shape).

Interview Answer

"DTOs represent the API's request/response shape, kept deliberately separate from internal domain/entity objects. This isn't just style — it's a real security control preventing mass assignment (a client setting fields they shouldn't via extra JSON properties an entity-bound endpoint would naively accept), and it decouples the API contract from internal schema changes. A dedicated response DTO with a single, centralized mapping method is a stronger guarantee against data leaks than relying on `@JsonIgnore` scattered across a shared entity."

Cross Questions

- Q: What is mass assignment, and how does using a DTO prevent it? → A: A vulnerability where a client sets unintended fields by including extra JSON properties in a request; a request DTO with a deliberately narrow set of fields structurally can't bind extra properties the client shouldn't control, since Jackson has nowhere to put them.
- Q: Why is centralizing entity-to-DTO mapping in one method preferable to inline mapping at every endpoint? → A: Consistency and correctness — a sensitive field's exclusion (or a formatting rule) only needs to be right in one place, rather than trusted to be remembered correctly at every individual call site.
- Q: How do DTOs help API evolution independent of the database schema? → A: The DTO's shape is decoupled from the entity's actual fields/column names — an internal schema change (renaming a column, restructuring a table) doesn't automatically break the public API contract, as long as the mapping method is updated to compensate.

Tricky Questions

- Q: If `CreateTaskRequest` accidentally DID include a `completed` field, would that reintroduce the mass-assignment risk even with a DTO? → A: Yes — DTOs only protect against mass assignment for fields they deliberately don't declare; a DTO carelessly designed with too many client-writable fields (including ones that should be server-controlled) still carries the same risk, so DTO field selection itself must be deliberate.
- Q: Is it ever acceptable to use the same class for both request and response DTOs? → A: It's generally discouraged for exactly the reasons above — a request DTO's job is defining what a client MAY send (often a narrower set), while a response DTO's job is defining what the client MAY see (often a different, sometimes broader, set); conflating them tends to reintroduce either mass-assignment risk or accidental data leaks.

Coding Exercise

L1: Build a request DTO and response DTO for a `Product` resource, with a centralized entity-to-DTO mapping method. **L2:** Reproduce the mass-assignment risk by binding a request directly to an entity with extra fields, then fix it with a dedicated request DTO. **L3:** Add a field to your internal entity that should never appear in the API response, and verify the response DTO correctly excludes it. **L4 (Interview):** Explain mass assignment and how request DTOs prevent it. **L5 (Senior):** Review an API binding `@RequestBody` directly to a JPA entity with an `isAdmin` field — explain the vulnerability and

the DTO-based fix. **L6 (Mastery):** Explain, from memory, why dedicated DTOs are a stronger guarantee against data leaks than `@JsonIgnore` alone (Book 10, Ch.6).

CHAPTER 5 — Validation with Bean Validation

Telugu Explanation

Bean Validation (`jakarta.validation` — `@NotNull` , `@NotBlank` , `@Size` , `@Min / @Max` , `@Email` , `@Pattern`) declarative గా input constraints define చేయడానికి వాడతారు. `@Valid` controller method parameter మీద పెడితే, Spring automatic గా request body ని validate చేసి, violations ఉంటే `MethodArgumentNotValidException` throw చేస్తుంది — దాన్ని Ch.6's global exception handler catch చేసి 400 Bad Request గా convert చేస్తుంది.

Professional English Explanation

Bean Validation (`jakarta.validation` — `@NotNull` , `@NotBlank` , `@Size` , `@Min / @Max` , `@Email` , `@Pattern`) declaratively defines input constraints. Placing `@Valid` on a controller method parameter makes Spring automatically validate the request body, throwing `MethodArgumentNotValidException` on violations — caught by Ch.6's global exception handler and converted to `400 Bad Request` .

Java Code

```
import jakarta.validation.constraints.*;
import org.springframework.web.bind.annotation.*;
import org.springframework.validation.annotation.Validated;
import org.springframework.http.ResponseEntity;

record CreateTaskRequest(
    @NotBlank(message = "Title is required and cannot be blank")
    @Size(max = 200, message = "Title cannot exceed 200 characters")
    String title,

    @NotNull(message = "Priority is required")
    @Min(value = 1, message = "Priority must be at least 1")
    @Max(value = 5, message = "Priority cannot exceed 5")
    Integer priority,

    @Email(message = "Must be a valid email address")
    String assigneeEmail
) {}

@RestController
@RequestMapping("/api/v3/tasks")
@Validated // enables validation for
@RequestParam/@PathVariable too
class ValidatedTaskController {

    @PostMapping
    ResponseEntity<String> create(@Valid @RequestBody CreateTaskRequest request) { //
@Valid TRIGGERS validation
        // If we reach this line, ALL constraints already passed - no manual if-checks needed
        return ResponseEntity.status(201).body("Created: " + request.title());
    }

    @GetMapping
    ResponseEntity<String> search(
        @RequestParam @Min(1) int page, //
        validates query params too
        @RequestParam @Size(min = 2, max = 50) String query) {
        return ResponseEntity.ok("Searching page " + page + " for: " + query);
    }
}
```

Sample Interaction

```
POST /api/v3/tasks
Body: {"title": "", "priority": 10, "assigneeEmail": "not-an-email"}
-> 400 Bad Request (once Ch.6's global handler processes MethodArgumentNotValidException)
{
  "errors": [
    "title: Title is required and cannot be blank",
    "priority: Priority cannot exceed 5",
    "assigneeEmail: Must be a valid email address"
  ]
}

POST /api/v3/tasks
Body: {"title": "Review PR", "priority": 3, "assigneeEmail": "ravi@example.com"}
-> 201 Created
Created: Review PR
```

Internal Working

- `@Valid` triggers Spring's integration with the **Bean Validation** specification (implemented by Hibernate Validator by default) — before the controller method body ever executes, every constraint annotation on the DTO's fields is checked; if **any** fail, Spring throws `MethodArgumentNotValidException` **before your method body runs at all** — this is precisely why the controller method in the demo can simply trust `request` is fully valid the moment execution reaches its first line, with zero manual `if` checks needed.
- Constraint annotations are themselves a form of **declarative, metadata-driven validation** — similar in spirit to Book 10, Ch.6's Jackson annotations (`@JsonProperty` , etc.) — the validation *logic* lives once in the Bean Validation framework/Hibernate Validator, and your DTOs simply *declare* which rules apply to which fields, rather than hand-writing imperative `if (title == null || title.isBlank()) throw ...` checks repeated across every endpoint.
- `@Validated` at the class level (as opposed to `@Valid` on a specific parameter) is specifically needed to enable constraint validation on **simple parameters** (`@RequestParam` / `@PathVariable`) rather than just request bodies — a genuinely different, less commonly known validation entry point than `@Valid` on a `@RequestBody` DTO.

Real-World Example

Telugu: Real production APIs లో input validation ప్రతి field కి manual `if` checks రాయడం బదులు, DTO మేరీద constraint annotations పెట్టడం ద్వారా — code చాలా concise గా, consistent గా, error-prone కాకుండా ఉంటుంది. ఇది Book 04's exception handling philosophy తో direct గా కలుస్తుంది — validation failures ఒక expected, structured error condition. English: Real production APIs express input validation via constraint annotations on DTOs rather than manual `if` checks scattered per field — dramatically more concise, consistent, and less error-prone. This connects directly to Book 04's exception-handling philosophy — a validation failure is an expected, structured error condition, cleanly handled via the exception mechanism rather than ad-hoc conditional logic.

Interview Answer

"Bean Validation lets you declaratively define input constraints (`@NotBlank` , `@Size` , `@Min / @Max` , `@Email`) directly on DTO fields. `@Valid` on a controller parameter triggers automatic validation before the method body runs — any violation throws `MethodArgumentNotValidException` , which a global exception handler (Ch.6) converts to a structured 400 response. `@Validated` at the class level additionally enables validation on simple `@RequestParam / @PathVariable` parameters, not just request bodies."

Cross Questions

- Q: When does validation happen relative to the controller method body executing? → A: Before — if `@Valid` validation fails, `MethodArgumentNotValidException` is thrown and the controller method body never executes at all.
- Q: Why is `@Validated` needed at the class level in addition to `@Valid`? → A: `@Valid` alone handles request-body validation; `@Validated` at the class level is what additionally enables constraint checking on simple `@RequestParam / @PathVariable` method parameters.
- Q: What's the practical benefit of declarative constraint annotations over manual `if` validation checks? → A: Conciseness, consistency (the same well-tested validation logic applies everywhere a given annotation is used), and reduced risk of a forgotten or subtly-wrong manual check on some endpoint.

Tricky Questions

- Q: Does `@NotNull` on an `Integer` field also reject an empty string `""` sent as JSON? → A: No — `@NotNull` only rejects an actual JSON `null` ; an empty string wouldn't even successfully deserialize into an `Integer` field in the first place (a different kind of error, a JSON parsing/type-mismatch failure, not a Bean Validation constraint violation).
- Q: Can custom, domain-specific validation logic (not covered by standard annotations) be added to this same mechanism? → A: Yes — Bean Validation supports custom constraint annotations (implementing `ConstraintValidator`) for domain-specific rules not covered by built-ins, integrating seamlessly with the same `@Valid` trigger mechanism.

Coding Exercise

L1: Add `@NotBlank` , `@Size` , and `@Min / @Max` constraints to a request DTO and test both valid and invalid inputs. **L2:** Add `@Validated` at the controller class level and validate a `@RequestParam` with `@Min` . **L3:** Research (via documentation) how to write a custom constraint annotation for a domain-specific rule (e.g., a valid product SKU format). **L4 (Interview):** Explain when `@Valid` validation happens relative to the controller method body. **L5 (Senior):** Review an API using manual `if` validation checks scattered across 10 endpoints — propose migrating to declarative Bean Validation and estimate the boilerplate reduction. **L6 (Mastery):** Explain, from memory, the difference between what `@Valid` and `@Validated` each enable.

CHAPTER 6 — Global Exception Handling

Telugu Explanation

Book 04, Ch.8 లో మనం plain Java లో centralized exception handling pattern build చేశాము. Spring Boot దీన్ని `@RestControllerAdvice` + `@ExceptionHandler` annotations తో formalize చేస్తుంది — ఒకే class, application అంతటా exception-to-response mapping decide చేస్తుంది, controller code business logic మాత్రమే రాస్తుంది.

Professional English Explanation

Book 04, Ch.8 built a centralized exception-handling pattern in plain Java. Spring Boot formalizes this via `@RestControllerAdvice` + `@ExceptionHandler` annotations — one class decides the exception-to-response mapping for the entire application, letting controller code focus purely on business logic.

Java Code

```
import org.springframework.web.bind.annotation.*;
import org.springframework.http.*;
import org.springframework.web.bind.MethodArgumentNotValidException;
import java.time.Instant;
import java.util.*;
import java.util.stream.Collectors;

// Domain-specific exceptions (Book 04, Ch.6's custom exception design)
class TaskNotFoundException extends RuntimeException {
    TaskNotFoundException(String id) { super("Task not found: " + id); }
}
class InvalidTaskStateException extends RuntimeException {
    InvalidTaskStateException(String message) { super(message); }
}

record ErrorResponse(int status, String error, String message, List<String> details, String
timestamp) {}

@RestControllerAdvice // applies to ALL
@RestController classes application-wide
class GlobalExceptionHandler {

    @ExceptionHandler(TaskNotFoundException.class)
    ResponseEntity<ErrorResponse> handleNotFound(TaskNotFoundException ex) {
        return buildResponse(HttpStatus.NOT_FOUND, ex.getMessage(), null); // 404
    }

    @ExceptionHandler(InvalidTaskStateException.class)
    ResponseEntity<ErrorResponse> handleInvalidState(InvalidTaskStateException ex) {
        return buildResponse(HttpStatus.CONFLICT, ex.getMessage(), null); // 409
    }

    @ExceptionHandler(MethodArgumentNotValidException.class) //
    Ch.5's validation failures land here
    ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        List<String> details = ex.getBindingResult().getFieldErrors().stream()
            .map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
            .collect(Collectors.toList()); //
        Book 07, Ch.7
        return buildResponse(HttpStatus.BAD_REQUEST, "Validation failed", details); //
        400
    }

    @ExceptionHandler(Exception.class) //
    FALLBACK - never leak internals
    ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        // In real code: log.error("Unexpected error", ex); - log FULL detail internally
        (Ch.9)
        return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR, "An unexpected error occurred",
        null); // 500, generic message
    }

    private ResponseEntity<ErrorResponse> buildResponse(HttpStatus status, String message,
    List<String> details) {
        ErrorResponse body = new ErrorResponse(status.value(), status.getReasonPhrase(),
        message, details,
            Instant.now().toString());
        return ResponseEntity.status(status).body(body);
    }
}
```

```

    }
}

@RestController
@RequestMapping("/api/v4/tasks")
class BusinessLogicOnlyController { // NOTICE: no try-catch
    anywhere - pure business logic!
    private final Map<String, String> tasks = new HashMap<>(Map.of("1", "Buy groceries"));

    @GetMapping("/{id}")
    String getTask(@PathVariable String id) {
        String task = tasks.get(id);
        if (task == null) throw new TaskNotFoundException(id); // just THROW -
        GlobalExceptionHandler handles the rest
        return task;
    }

    @PostMapping("/{id}/complete")
    String completeTask(@PathVariable String id) {
        if (!tasks.containsKey(id)) throw new TaskNotFoundException(id);
        throw new InvalidTaskStateException("Task " + id + " is already completed"); //
        simulated business rule
    }
}

```

Sample Interaction

```

GET /api/v4/tasks/999
-> 404 Not Found
{"status":404,"error":"Not Found","message":"Task not found:
999","details":null,"timestamp":"2026-08-28T..."}

POST /api/v4/tasks/1/complete
-> 409 Conflict
{"status":409,"error":"Conflict","message":"Task 1 is already
completed","details":null,"timestamp":"..."}

```

Internal Working

- `@RestControllerAdvice` is itself implemented via Spring's AOP-adjacent interception machinery (Book 11, Ch.8) at the `DispatcherServlet` level — when a controller method throws an exception that propagates out uncaught (Book 04, Ch.4's propagation), `DispatcherServlet` looks for a matching `@ExceptionHandler` method across all registered `@RestControllerAdvice` classes, using the **most specific matching exception type** (exactly like Book 01's IS-A polymorphic catch matching) — this is precisely why `BusinessLogicOnlyController`'s methods need zero try-catch blocks at all.
- The **fallback** `@ExceptionHandler(Exception.class)` is critical, directly implementing Book 04, Ch.8's "never leak internals to external clients" principle — any truly unanticipated exception is caught here, logged with full detail internally, but only a generic, safe message is returned to the client, exactly the pattern Book 04's plain-Java demo established.
- This chapter's `GlobalExceptionHandler` is **structurally identical** to Book 04, Ch.8's plain-Java `GlobalExceptionHandler.handle(Throwable)` — Spring Boot didn't invent a new concept, it formalized exactly that pattern with annotations, automatic exception-type-based dispatch

(instead of manual `instanceof` checks), and automatic JSON serialization of the resulting `ErrorResponse` .

Real-World Example

Telugu: ఈ chapter యొక్క `@RestControllerAdvice` ఖచ్చితంగా Book 04, Ch.8 లో మనం "OrderController"/"GlobalExceptionHandler" నా plain Java లో design చేసిన pattern యొక్క Spring-native, annotation-driven version — controller code business logic మాత్రమే, error formatting అంతా centralized. English: This chapter's `@RestControllerAdvice` is exactly the Spring-native, annotation-driven version of the pattern designed in plain Java back in Book 04, Ch.8's `OrderController / GlobalExceptionHandler` — controller code stays pure business logic, error formatting stays fully centralized.

Interview Answer

"`@RestControllerAdvice` with `@ExceptionHandler` methods centralizes exception-to-response mapping across the entire application, letting controllers simply throw meaningful domain exceptions with zero try-catch. Spring matches the most specific `@ExceptionHandler` for a propagating exception, exactly like polymorphic exception matching in plain Java (Book 04). A fallback `ExceptionHandler` handler ensures unanticipated errors never leak internal details to clients while still being fully logged — this formalizes exactly the pattern built manually in Book 04, Ch.8."

Cross Questions

- Q: Do controller methods in a well-designed Spring Boot app need try-catch blocks for expected business exceptions? → A: Generally no — they simply throw meaningful exceptions and let them propagate; the centralized `@RestControllerAdvice` handles the exception-to-response translation.
- Q: How does Spring decide which `@ExceptionHandler` method to use for a given exception? → A: The most specific matching exception type registered across all `@RestControllerAdvice` classes — the same polymorphic type-matching principle as a regular `catch` block (Book 04, Ch.1).
- Q: Why is the fallback `ExceptionHandler` handler important? → A: It ensures truly unanticipated exceptions still produce a safe, generic client-facing response instead of leaking a stack trace or internal details, while still being fully logged internally for debugging.

Tricky Questions

- Q: If both a `TaskNotFoundException` -specific handler and the generic `ExceptionHandler` handler exist, which one handles a thrown `TaskNotFoundException` ? → A: The more specific one (`TaskNotFoundException` 's own handler) — Spring's matching, like polymorphic catch resolution, always prefers the most specific applicable type.
- Q: Does `@RestControllerAdvice` apply only to controllers in the same package, or application-wide by default? → A: Application-wide by default, across every `@RestController` in the application — it can optionally be scoped narrower (to specific packages or annotation types) via attributes, but the default is global.

Coding Exercise

L1: Build a `GlobalExceptionHandler` handling 2 custom exceptions plus a generic fallback, and a controller throwing them without any try-catch. **L2:** Add the `MethodArgumentNotValidException` handler from this chapter and verify it correctly reports Ch.5's validation errors. **L3:** Verify (via logging) that the fallback handler logs full exception detail internally while returning only a generic message externally. **L4 (Interview):** Explain how `@RestControllerAdvice` connects to Book 04, Ch.8's plain-Java centralized exception handling pattern. **L5 (Senior):** Design a complete exception hierarchy and `@RestControllerAdvice` for an e-commerce API (out-of-stock, payment declined, invalid coupon), mapping each to the correct HTTP status. **L6 (Mastery):** Explain, from memory, why the fallback `ExceptionHandler` handler is a security-relevant piece of the design, not just a catch-all convenience.

CHAPTER 7 — Service & Repository Layers

Telugu Explanation

Book 10, Ch.9's layered architecture ఇప్పుడు Spring annotations తో: `@Service` (business logic, HTTP-agnostic), `@Repository` (data access — Book 09's JDBC, లేదా Spring Data JPA, Book 13లో పూర్తిగా). ఈ chapter Spring Data JPA repository interface ని ఒక **preview** గా చూపిస్తుంది — పూర్తి depth Book 13.

Professional English Explanation

Book 10, Ch.9's layered architecture, now with Spring annotations: `@Service` (business logic, HTTP-agnostic), `@Repository` (data access — Book 09's JDBC, or Spring Data JPA, covered fully in Book 13). This chapter previews a Spring Data JPA repository interface — full depth in Book 13.

Java Code

```
import org.springframework.stereotype.*;
import org.springframework.data.jpa.repository.JpaRepository;           // Book 13 covers
this fully
import jakarta.persistence.*;
import java.util.*;

@Entity                                                              // Book 13 - JPA entity, maps
to a DB table
class Task {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;
    String title;
    boolean completed;
    Task() {}                                                         // no-arg constructor -
    // required by JPA (Book 01, Ch.8)
    Task(String title) { this.title = title; }
}

interface TaskRepository extends JpaRepository<Task, Long> {         // Spring Data JPA - Book
    // 13's Repository pattern (Book 06)
    List<Task> findByCompletedFalse();                               // METHOD NAME becomes
    // the query - Book 13 magic
}

@Service                                                            // business logic,
// HTTP-agnostic (Book 10, Ch.9)
class TaskBusinessService {
    private final TaskRepository repository;

    TaskBusinessService(TaskRepository repository) {                 // constructor
    // injection (Book 11, Ch.5)
        this.repository = repository;
    }

    Task createTask(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Title required");     // business
            // rule, HTTP-agnostic
        }
        return repository.save(new Task(title));
    }

    Task completeTask(Long id) {
        Task task = repository.findById(id)
            .orElseThrow(() -> new TaskNotFoundException(String.valueOf(id))); //
        // Optional (Book 07, Ch.9)
        task.completed = true;
        return repository.save(task);                                 // JPA's
        // "save = insert or update"
    }

    List<Task> getIncompleteTasks() { return repository.findByCompletedFalse(); }
}

@RestController
@RequestMapping("/api/v5/tasks")
class LayeredTaskController {                                       // pure
    // HTTP translation, Ch.6's exceptions
}
```

```

private final TaskBusinessService service;
LayeredTaskController(TaskBusinessService service) { this.service = service; }

@PostMapping
org.springframework.http.ResponseEntity<Task>
create(@org.springframework.web.bind.annotation.RequestBody String title) {
    Task task = service.createTask(title);
    return org.springframework.http.ResponseEntity.status(201).body(task);
}

@GetMapping("/incomplete")
List<Task> incomplete() { return service.getIncompleteTasks(); }
}

```

Internal Working

- `TaskRepository` extends `JpaRepository<Task, Long>` is a genuinely remarkable piece of Spring Data machinery, fully explained in Book 13: it's an **interface with no implementation code at all**, yet Spring generates a full, working implementation at runtime (via a JDK dynamic proxy, Book 11, Ch.8-adjacent mechanism) providing `save()`, `findById()`, `findAll()`, `deleteById()` automatically, PLUS custom finder methods like `findByCompletedFalse()` derived purely from the **method name itself** — Spring parses the name (`findBy` + `Completed` + `False`) into a query, with zero SQL/JSQL written by you for this simple case.
- The `TaskBusinessService` layer here is **exactly** Book 10, Ch.9's Service layer principle in action — it depends on `TaskRepository` (an abstraction, per Book 02's DIP), knows nothing about HTTP, and throws plain business exceptions (`IllegalArgumentException`, the custom `TaskNotFoundException`) that Ch.6's `@RestControllerAdvice` translates to HTTP responses — the controller layer above it is reduced to pure HTTP-translation glue.
- `repository.save(task)` for an entity that already has an `id` set performs an **UPDATE**, not an INSERT — JPA's `save()` is a unified "insert-or-update" operation (technically, "merge" semantics), a genuinely important behavioral detail fully explored with all its nuances in Book 13.

Real-World Example

Telugu: ఈ chapter యొక్క `TaskRepository / TaskBusinessService / LayeredTaskController` structure ఖచ్చితంగా real production Spring Boot applications యొక్క standard shape — Book 09 (JDBC), Book 10 (layered architecture), Book 11 (DI), ఇప్పుడు అన్నీ ఒక్కచోట కలిసి, Book 13 (JPA) కి direct bridge గా పనిచేస్తాయి. English: This chapter's `TaskRepository / TaskBusinessService / LayeredTaskController` structure is exactly the standard shape of real production Spring Boot applications — Book 09 (JDBC), Book 10 (layered architecture), and Book 11 (DI) all converge here, serving as a direct bridge into Book 13's full JPA depth.

Interview Answer

"The Service layer (`@Service`) holds HTTP-agnostic business logic, depending on Repository abstractions per Book 02's DIP, and throws domain exceptions that a `@RestControllerAdvice` (Ch.6) translates to HTTP responses. The Repository layer (`@Repository`, or a Spring Data `JpaRepository` interface) handles data access — Spring Data JPA can generate a full working implementation from

just an interface declaration, including custom finder methods derived from the method name itself, with zero implementation code written by the developer for common cases."

Cross Questions

- Q: Does `TaskRepository` (an interface extending `JpaRepository`) need any implementation code written by the developer? → A: No, for standard CRUD operations and simple derived-query methods — Spring generates a working implementation automatically at runtime; custom complex queries can still be added via `@Query` (Book 13).
- Q: What does `repository.save(task)` do if `task.id` is already set to an existing value? → A: An UPDATE of the existing row (merge semantics), not a duplicate INSERT — JPA's `save()` unifies insert-or-update behavior.
- Q: Why does `TaskBusinessService` throw plain exceptions instead of building HTTP responses itself? → A: To stay HTTP-agnostic, per Book 10, Ch.9's layered architecture principle — HTTP-specific translation belongs in the Controller layer (or the centralized `@RestControllerAdvice`, Ch.6), not the Service layer.

Tricky Questions

- Q: How does Spring Data JPA derive a query from a method name like `findByCompletedFalse()`? → A: It parses the method name into a structured query template (`SELECT ... WHERE completed = false`) based on a documented naming convention (`findBy<FieldName><Condition>`), covered fully in Book 13 — this works for simple field-based conditions without any SQL/JPQL written explicitly.
- Q: If `TaskRepository` were a plain `@Repository`-annotated class using raw JDBC (Book 09) instead of `JpaRepository`, would the Service layer above it need to change at all? → A: No — this is precisely the DIP payoff (Book 02): the Service layer depends only on the `TaskRepository` abstraction's method signatures, not its implementation technology, so swapping JDBC for JPA underneath doesn't require touching the Service or Controller layers at all.

Coding Exercise

L1: Build a `@Service / Repository` pair for a `Product` domain, with the Service layer throwing plain business exceptions. **L2:** Add a Spring Data derived-query method (e.g., `findByPriceLessThan(double price)`) and use it from the Service layer. **L3:** Verify that calling `save()` on an entity with an existing ID updates rather than duplicates the row. **L4 (Interview):** Explain why the Service layer should never build HTTP responses directly. **L5 (Senior):** Design the layered structure (Controller/Service/Repository/Entity) for a new `Order` feature, specifying each layer's exact responsibility. **L6 (Mastery):** Explain, from memory, how Spring Data JPA can provide a full repository implementation from just an interface, connecting it to Book 11's dynamic-proxy-based bean mechanics.

CHAPTER 8 — Configuration Management & Profiles

Telugu Explanation

`application.properties` / `application.yml` externalized configuration కి standard location.
`@ConfigurationProperties` ఒక group of related properties ని ఒక్క, type-safe object గా bind చేస్తుంది (individual `@Value` calls కంటే cleaner). Profile-specific files (`application-dev.properties` , `application-prod.properties`) Book 11, Ch.7's `@Profile` కి direct file-naming convention.

Professional English Explanation

`application.properties` / `application.yml` is the standard location for externalized configuration.
`@ConfigurationProperties` binds a group of related properties into a single, type-safe object (cleaner than individual `@Value` calls). Profile-specific files (`application-dev.properties` , `application-prod.properties`) are the direct file-naming convention for Book 11, Ch.7's `@Profile`.

Configuration Files

```
# application.properties (base config, always loaded)
spring.application.name=task-manager
server.port=8080
logging.level.root=INFO

app.task.default-priority=3
app.task.max-title-length=200
app.task.notification-enabled=true
```

```
# application-dev.properties (only loaded when 'dev' profile is active)
spring.datasource.url=jdbc:h2:mem:devdb
logging.level.com.example.taskmanager=DEBUG
app.task.notification-enabled=false
```

```
# application-prod.properties (only loaded when 'prod' profile is active)
spring.datasource.url=${DATABASE_URL}
spring.datasource.username=${DATABASE_USER}
spring.datasource.password=${DATABASE_PASSWORD}
logging.level.com.example.taskmanager=WARN
```

Java Code

```
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.context.annotation.*;
import org.springframework.stereotype.Component;

@Component
@ConfigurationProperties(prefix = "app.task")           // binds ALL "app.task.*"
properties to this ONE object
class TaskProperties {
    private int defaultPriority;
    private int maxTitleLength;
    private boolean notificationEnabled;

    // getters and setters required for property binding (or use a constructor-binding record
    in newer Spring Boot)
    public int getDefaultPriority() { return defaultPriority; }
    public void setDefaultPriority(int v) { this.defaultPriority = v; }
    public int getMaxTitleLength() { return maxTitleLength; }
    public void setMaxTitleLength(int v) { this.maxTitleLength = v; }
    public boolean isNotificationEnabled() { return notificationEnabled; }
    public void setNotificationEnabled(boolean v) { this.notificationEnabled = v; }
}

@Component
class TaskConfigDemo {
    private final TaskProperties properties;
    TaskConfigDemo(TaskProperties properties) { this.properties = properties; }           // type-
    safe, no string keys anywhere

    void printConfig() {
        System.out.println("Default priority: " + properties.getDefaultPriority());
        System.out.println("Max title length: " + properties.getMaxTitleLength());
        System.out.println("Notifications enabled: " + properties.isNotificationEnabled());
    }
}
```

Internal Working

- `@ConfigurationProperties(prefix = "app.task")` binds via **reflection** (matching property key suffixes to setter method names — `app.task.default-priority` → `setDefaultPriority()`, following a kebab-case-to-camelCase relaxed binding convention) — this eliminates the repetitive `@Value("${app.task.default-priority}")` on every individual field, and critically provides **type safety and IDE autocomplete** for configuration access throughout the codebase, since you're now calling `properties.getDefaultPriority()` (a real method) rather than referencing a magic string key that could be typo'd with no compile-time check.
- Profile-specific files activate via `spring.profiles.active=dev` (in the base `application.properties`, an environment variable, or a command-line argument `--spring.profiles.active=prod`) — Spring Boot loads the base `application.properties` **first**, then layers the profile-specific file's properties **on top**, overriding any matching keys — this is exactly Book 11, Ch.7's `@Profile` bean-selection mechanism, extended to property-level configuration too.

- Using `${DATABASE_URL}` syntax in `application-prod.properties` references an **environment variable** — this is the standard, secure production pattern (Book 10, Ch.2's "never hardcode credentials" principle) for injecting secrets at deploy time (via container orchestration, a secrets manager, etc.) rather than committing them to version control in any properties file.

Real-World Example

Telugu: Real production Kubernetes deployments inject environment variables (`DATABASE_URL` , `DATABASE_PASSWORD`) as container config, with `spring.profiles.active=prod` కూడా environment variable నా set చేస్తారు — code మార్చకుండా, ఒకే JAR artifact dev/staging/prod అన్నింటిలో deploy అవుతుంది, ఒక్క environment variable మార్చడం ద్వారా. English: Real production Kubernetes deployments inject environment variables (`DATABASE_URL` , `DATABASE_PASSWORD`) as container config, with `spring.profiles.active=prod` also set as an environment variable — the same JAR artifact deploys unchanged across dev/staging/prod, differentiated purely by environment variables, exactly the pattern this chapter establishes.

Interview Answer

"`application.properties / .yaml` holds externalized configuration, with profile-specific files (`application-{profile}.properties`) layering on top of the base file when that profile is active — the file-naming extension of Book 11, Ch.7's `@Profile` . `@ConfigurationProperties` binds a group of related properties into one type-safe object via reflection, cleaner and safer than scattered `@Value` calls with string keys. Production secrets are injected via environment variable placeholders (`${DATABASE_URL}`), never hardcoded, following the same principle from Book 10."

Cross Questions

- Q: What's the advantage of `@ConfigurationProperties` over multiple individual `@Value` annotations? → A: Type safety and IDE support (real method calls instead of string-keyed lookups prone to typos with no compile-time check), and grouping related configuration into one coherent object.
- Q: How does a profile-specific properties file relate to the base `application.properties` ? → A: It's loaded on top of the base file when that profile is active, overriding any matching keys — the base file's other, non-overridden properties still apply.
- Q: Why use `${DATABASE_URL}` instead of hardcoding a production database URL directly in `application-prod.properties` ? → A: Security and flexibility — actual credentials/URLs are injected via environment variables at deploy time, never committed to version control, and can differ per actual deployment instance without any code/config file change.

Tricky Questions

- Q: If both the base `application.properties` and `application-prod.properties` define `logging.level.root` , which one wins when the `prod` profile is active? → A: The profile-specific file's value wins — profile-specific properties override the base file's values for matching keys.
- Q: Does `@ConfigurationProperties` require getters AND setters, or can it use constructor binding instead? → A: Modern Spring Boot supports **constructor binding** (especially clean with

an immutable Java `record` , Book 07 Ch.13) as an alternative to the getter/setter JavaBean style shown here — both are valid, with constructor binding often preferred for its immutability benefits (Book 02, Ch.15).

Coding Exercise

L1: Create a `@ConfigurationProperties` class binding a group of custom `app.*` properties, and inject/use it in a component. **L2:** Set up `dev` and `prod` profile-specific property files with different values for the same key, and switch between them. **L3:** Convert your `@ConfigurationProperties` class to use constructor binding with a record instead of getters/setters. **L4 (Interview):** Explain the advantage of `@ConfigurationProperties` over individual `@Value` annotations. **L5 (Senior):** Design a configuration strategy for a service needing different database URLs, log levels, and feature flags across dev/staging/prod, using profile-specific files and environment variable placeholders. **L6 (Mastery):** Explain, from memory, how profile-specific property files layer on top of the base `application.properties` .

CHAPTER 9 — Logging

Telugu Explanation

Spring Boot default గా **SLF4J** (a logging facade, actual implementation Logback) వాడుతుంది.

`System.out.println()` production code లో ఎప్పుడూ వాడకూడదు — structured logging (levels: TRACE, DEBUG, INFO, WARN, ERROR) వాడాలి, environment బట్టి (Ch.8's profiles) log verbosity control చేయడానికి, మరియు log aggregation tools (production monitoring) కి సరిగ్గా integrate అవ్వడానికి.

Professional English Explanation

Spring Boot uses **SLF4J** (a logging facade, with Logback as the actual implementation) by default.

`System.out.println()` should never be used in production code — structured logging (levels: TRACE, DEBUG, INFO, WARN, ERROR) is used instead, controllable per environment (Ch.8's profiles), and properly integrable with log aggregation tools for production monitoring.

Java Code

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

@Service
class LoggingDemoService {
    private static final Logger log = LoggerFactory.getLogger(LoggingDemoService.class);
    // one logger per class, by convention

    void processTask(String taskId, double amount) {
        log.debug("Starting to process task {}", taskId); // DEBUG -
        // detailed, dev-time only usually

        if (amount <= 0) {
            log.warn("Task {} has non-positive amount: {}", taskId, amount); // WARN
            // - suspicious but not fatal
        }

        try {
            log.info("Processing task {} with amount {}", taskId, amount); //
            // INFO - normal, notable events
            if (amount > 1_000_000) throw new IllegalStateException("Amount too large");
        } catch (Exception e) {
            log.error("Failed to process task {}", taskId, e); //
            // ERROR - pass the exception ITSELF, not just its message!
            throw e;
        }

        log.trace("Task {} processing details: amount={}", taskId, amount); //
        // TRACE - most granular, rarely enabled
    }
}
```

application.properties — Log Level Configuration

```
# Root level applies to everything not more specifically configured
logging.level.root=INFO

# Package-specific override - DEBUG for YOUR code, but keep noisy framework logs at INFO
logging.level.com.example.taskmanager=DEBUG
logging.level.org.springframework=INFO
logging.level.org.hibernate.SQL=DEBUG    # useful for seeing actual SQL queries (Book 13)
during development

# Structured output pattern (timestamp, level, logger, message)
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} %-5level [%thread] %logger{36} - %msg%n
```

Internal Working

- **SLF4J is a facade, not an implementation** — your code depends only on the `Logger / LoggerFactory` interface, never a specific logging library directly; Spring Boot wires in Logback underneath by default, but the actual implementation could be swapped (Log4j2, java.util.logging) without changing a single line of your logging code — this is Book 02, Ch.13's Dependency Inversion Principle applied to logging infrastructure itself.
- **`log.error("...", e)`** (passing the exception object as the final argument) is critical, not optional — this causes the logging framework to capture and print the **full stack trace**, essential for real debugging (Book 04's exception-handling philosophy); `log.error("Failed: " + e.getMessage())` (just the message, string-concatenated) discards the stack trace entirely, a genuine, common, costly logging mistake that makes production debugging significantly harder.
- **Package-specific log level overrides** (`logging.level.com.example.taskmanager=DEBUG` while `org.springframework=INFO`) let you get verbose visibility into *your own* code without being flooded by framework-internal noise — a real, practical production/development tuning technique, and directly connects to Ch.8's profile-specific configuration (dev might enable DEBUG broadly; prod restricts to WARN/ERROR for performance and noise reasons).

Real-World Example

Telugu: Production incident debug చేసేటప్పుడు, `log.error()` కి exception object పాస్ చేయకపోతే, stack trace పోతుంది — root cause కనుక్కోవడం చాలా కష్టం అవుతుంది. ఇది Book 04, Ch.7's "losing the original cause" anti-pattern యొక్క logging-specific version. English: During production incident debugging, forgetting to pass the exception object to `log.error()` loses the stack trace entirely — making root-cause diagnosis significantly harder. This is exactly the logging-specific manifestation of Book 04, Ch.7's "losing the original cause" anti-pattern.

Interview Answer

"Spring Boot uses SLF4J as a logging facade (implementation-agnostic, per Book 02's DIP), with Logback as the default implementation. Structured logging with levels (TRACE/DEBUG/INFO/WARN/ERROR) replaces `System.out.println()` in production code, controllable per-package and per-profile. A critical, common mistake is calling `log.error(message)`

with just a string instead of `log.error(message, exception)` — the latter is required to actually capture and print the stack trace, essential for real debugging."

Cross Questions

- Q: Why is SLF4J described as a "facade"? → A: Your code depends only on SLF4J's `Logger / LoggerFactory` API, not a specific logging implementation — the actual backend (Logback by default in Spring Boot, or Log4j2) can be swapped without changing application code, exactly the Dependency Inversion Principle (Book 02) applied to logging.
- Q: What's the practical consequence of calling `log.error("Failed: " + e.getMessage())` instead of `log.error("Failed", e)`? → A: The full stack trace is lost — only the exception's message string is logged, making root-cause diagnosis in production significantly harder, since you lose exactly *where* in the code the exception originated.
- Q: Why would you set `logging.level.com.example.taskmanager=DEBUG` while leaving `org.springframework=INFO`? → A: To get detailed visibility into your own application's behavior without being flooded by verbose framework-internal logging noise you don't need for typical debugging.

Tricky Questions

- Q: Does `log.debug("Processing {}", taskId)` (with the `{}` placeholder) always construct the full formatted string, even if DEBUG level is disabled? → A: No — this is a deliberate, real performance optimization: SLF4J's parameterized logging only performs the actual string formatting if that log level is enabled, avoiding wasted string-concatenation work for disabled log statements (unlike naive `"Processing " + taskId` concatenation, which always executes regardless of whether the log level is enabled).
- Q: Is it safe to log an entire request/response body containing user PII or credentials at INFO level in production? → A: No — this is a real, documented data-handling/compliance risk; sensitive data should never be logged in plaintext (or at all, in many regulated contexts), a genuine production security consideration beyond just "more logging is better."

Coding Exercise

L1: Add SLF4J logging at multiple levels (DEBUG, INFO, WARN, ERROR) to a service class, and configure package-specific log levels. **L2:** Reproduce the "lost stack trace" mistake (`log.error(msg + e.getMessage())`) versus the correct form (`log.error(msg, e)`), and compare the output. **L3:** Verify (conceptually or via benchmarking) that parameterized logging (`log.debug("{} ", value)`) avoids string construction when DEBUG is disabled. **L4 (Interview):** Explain why SLF4J is called a facade and what benefit that provides. **L5 (Senior):** Design a logging strategy (levels, what to log, what never to log) for a payment-processing service handling sensitive customer data. **L6 (Mastery):** Explain, from memory, why `log.error(message, exception)` is critical for production debugging, connecting it to Book 04, Ch.7's "losing the cause" anti-pattern.

CHAPTER 10 — Spring Boot Actuator

Telugu Explanation

Spring Boot Actuator production applications ని **monitor** చేయడానికి, **manage** చేయడానికి ready-made endpoints అందిస్తుంది: `/actuator/health` (application healthy గా ఉందా), `/actuator/metrics` (JVM memory, Book 03; request counts; custom metrics), `/actuator/info` (build/version info), `/actuator/env` (active configuration — sensitive గా treat చేయాలి).

Professional English Explanation

Spring Boot Actuator provides ready-made endpoints for **monitoring** and **managing** production applications: `/actuator/health` (is the application healthy), `/actuator/metrics` (JVM memory, Book 03; request counts; custom metrics), `/actuator/info` (build/version info), `/actuator/env` (active configuration — must be treated as sensitive).

Java Code & Configuration

```
# application.properties
management.endpoints.web.exposure.include=health,metrics,info    # explicitly opt-in - NEVER
expose everything by default
management.endpoint.health.show-details=when-authorized        # don't leak internal
detail to unauthenticated callers
```

```
import org.springframework.boot.actuate.health.*;
import org.springframework.stereotype.Component;

@Component
class DatabaseHealthIndicator implements HealthIndicator {           // CUSTOM health check,
integrates into /actuator/health

    @Override
    public Health health() {
        boolean databaseReachable = checkDatabaseConnection();      // Book 09's
        Connection.isValid()
        if (databaseReachable) {
            return Health.up().withDetail("database", "reachable").build();
        }
        return Health.down().withDetail("database", "unreachable").build(); // DOWN status
        propagates to overall health
    }

    private boolean checkDatabaseConnection() {
        return true;                                                 // simulated
    }
    - real impl uses Book 09's Connection
}
```

Sample Actuator Responses

```
GET /actuator/health
-> 200 OK
{"status": "UP", "components": {"database": {"status": "UP", "details": {"database": "reachable"}}}}

GET /actuator/metrics/jvm.memory.used
-> 200 OK
{"name": "jvm.memory.used", "measurements": [{"statistic": "VALUE", "value": 134217728}]}
(this is literally Book 03's heap memory usage, exposed as a metric)

GET /actuator/info
-> 200 OK
{"app": {"name": "task-manager", "version": "1.0.0"}}
```

Internal Working

- Actuator endpoints are **not automatically all exposed** over HTTP by default — only `/actuator/health` and `/actuator/info` are web-exposed out of the box; everything else requires explicit opt-in via `management.endpoints.web.exposure.include`, a deliberate **secure-by-default** design, since endpoints like `/actuator/env` or `/actuator/heapdump` can leak significant internal detail (configuration values, even secrets if not carefully filtered) if exposed unauthenticated in production — this is a direct, practical application of Book 10's authorization principles to Spring's own tooling.
- A custom `HealthIndicator` bean is automatically **discovered and aggregated** by Actuator's overall `/actuator/health` endpoint — if your custom indicator reports `DOWN`, the overall application health rolls up to `DOWN` too, which is exactly the signal container orchestration systems (Kubernetes liveness/readiness probes, Book 16) use to decide whether to route traffic to, or restart, a given application instance.
- `/actuator/metrics` integrates with **Micrometer**, Spring's metrics-facade library (conceptually similar to SLF4J being a logging facade, Ch.9) — it can export to Prometheus, Datadog, and other real production monitoring backends, making Actuator the foundation of real production observability, not just a development convenience.

Real-World Example

Telugu: Kubernetes deployments `/actuator/health` ని **liveness/readiness probe** గా వాడతాయి — application unhealthy అయితే (ఉదా. database unreachable), Kubernetes automatic గా traffic ఆ instance కి routing ఆపేసి, restart కూడా చేయవచ్చు. ఇది production reliability కి directly contribute చేసే, extremely common real-world Actuator use case. English: Kubernetes deployments use `/actuator/health` as a **liveness/readiness probe** — if the application reports unhealthy (e.g., database unreachable), Kubernetes automatically stops routing traffic to that instance and can restart it — a directly reliability-critical, extremely common real-world Actuator use case.

Interview Answer

"Spring Boot Actuator provides production-ready monitoring/management endpoints — `/actuator/health`, `/actuator/metrics`, `/actuator/info`, and more. Only health and info are web-

exposed by default; everything else requires explicit opt-in, since endpoints can leak sensitive internal detail if exposed unauthenticated. Custom `HealthIndicator` beans integrate into the overall health rollup, which container orchestration systems (Kubernetes) use for liveness/readiness probes. Metrics integrate with Micrometer, exportable to real production monitoring backends like Prometheus."

Cross Questions

- Q: Are all Actuator endpoints exposed over HTTP by default? → A: No — only `/actuator/health` and `/actuator/info` are web-exposed out of the box; others require explicit opt-in via configuration, a secure-by-default design.
- Q: How does a custom `HealthIndicator` affect the overall `/actuator/health` response? → A: It's automatically discovered and its status contributes to the overall health rollup — if it reports DOWN, the overall application health becomes DOWN too.
- Q: What real-world infrastructure component commonly consumes `/actuator/health`? → A: Container orchestration systems like Kubernetes, using it for liveness/readiness probes to decide whether to route traffic to or restart an instance.

Tricky Questions

- Q: Why might exposing `/actuator/env` unauthenticated in production be a real security risk? → A: It can reveal active configuration values, potentially including sensitive ones (though Spring Boot does attempt to sanitize obviously-sensitive-looking keys), giving an attacker significant reconnaissance information about the application's internals — it should generally be secured (Book 14) or not exposed at all in production.
- Q: Does a slow (but eventually successful) database health check affect Actuator's usefulness as a Kubernetes readiness probe? → A: Yes, potentially — a health check itself needs to be fast and reliable, since a probe that times out or is slow can cause Kubernetes to make incorrect routing/restart decisions; health check implementation performance is itself a real production design consideration.

Coding Exercise

L1: Add the Actuator starter to a project and observe the default-exposed `/actuator/health` and `/actuator/info` endpoints. **L2:** Explicitly expose `/actuator/metrics` and inspect a JVM memory metric, connecting it to Book 03's memory model. **L3:** Write a custom `HealthIndicator` for a simulated external dependency, and verify it affects the overall health status. **L4 (Interview):** Explain why Actuator doesn't expose all endpoints by default. **L5 (Senior):** Design an Actuator configuration strategy for a production service, specifying which endpoints to expose, how to secure sensitive ones (Book 14 preview), and how Kubernetes would use the health endpoint. **L6 (Mastery):** Explain, from memory, how Micrometer relates to Actuator's metrics, connecting it to SLF4J's facade role from Ch.9.

CHAPTER 11 — Testing Spring Boot Applications (Preview)

Telugu Explanation

ఇది Book 15's full-depth testing topic కి ఒక **preview** — Spring Boot testing యొక్క 3 ప్రధాన layers: **Unit tests** (plain JUnit + Mockito, Spring context అవసరం లేదు — Book 11, Ch.5's constructor injection testability payoff ఇక్కడే), **@WebMvcTest** (controller layer మాత్రమే, **MockMvc** వాడి, HTTP-level testing), **@SpringBootTest** (full application context, integration testing).

Professional English Explanation

This is a preview of Book 15's full-depth testing coverage — Spring Boot testing has 3 main layers: **Unit tests** (plain JUnit + Mockito, no Spring context needed — this is exactly where Book 11, Ch.5's constructor-injection testability payoff is realized), **@WebMvcTest** (controller layer only, using **MockMvc** for HTTP-level testing), and **@SpringBootTest** (full application context, integration testing).

Java Code

```
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.boot.test.mock.mockito.MockBean;
import static org.junit.jupiter.api.Assertions.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

// LEVEL 1: Pure unit test - NO Spring context, fast, isolated (seconds, not context-startup
time)
class TaskBusinessServiceUnitTest {
    @Test
    void createTask_throwsException_whenTitleBlank() {
        TaskRepository mockRepo = Mockito.mock(TaskRepository.class);           // Book 11,
Ch.5's testability payoff
        TaskBusinessService service = new TaskBusinessService(mockRepo);       // plain
'new' - NO Spring context at all!

        assertThrows(IllegalArgumentException.class, () -> service.createTask(""));
        Mockito.verifyNoInteractions(mockRepo);                               //
repository never touched - fails fast
    }
}

// LEVEL 2: Controller-layer test - loads ONLY the web layer, mocks the service, fast HTTP-
level testing
@WebMvcTest(LayeredTaskController.class)
class LayeredTaskControllerWebTest {
    @org.springframework.beans.factory.annotation.Autowired
    private MockMvc mockMvc;                                                  //
simulates HTTP requests, no real server

    @MockBean
    private TaskBusinessService mockService;                                  //
FAKE service - controller tested in isolation

    @Test
    void getIncompleteTasks_returns200AndJson() throws Exception {
        Mockito.when(mockService.getIncompleteTasks())
            .thenReturn(java.util.List.of(new Task("Buy groceries")));

        mockMvc.perform(get("/api/v5/tasks/incomplete"))
            .andExpect(status().isOk())                                         //
Book 10, Ch.1's status codes
            .andExpect(content().contentType("application/json"))
            .andExpect(jsonPath("$[0].title").value("Buy groceries"));
    }
}

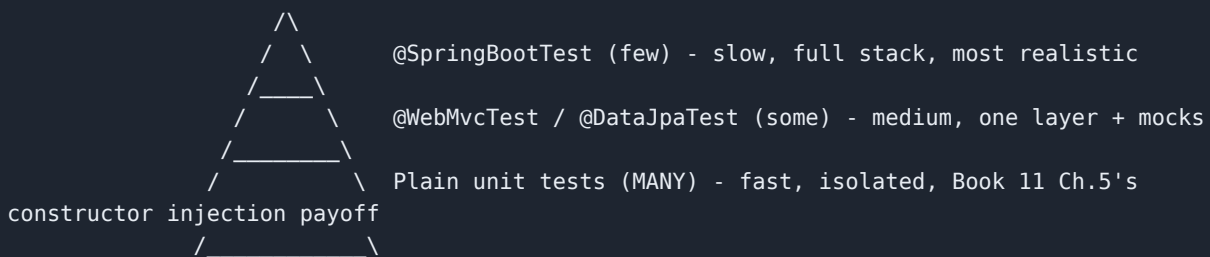
// LEVEL 3: Full integration test - REAL Spring context, REAL (in-memory) database, slowest but
most realistic
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class TaskManagerIntegrationTest {
    @org.springframework.beans.factory.annotation.Autowired
    private org.springframework.boot.test.web.client.TestRestTemplate restTemplate; //
makes REAL HTTP calls
```

```

@Test
void fullCreateAndRetrieveFlow() {
    var createResponse = restTemplate.postForEntity("/api/v5/tasks", "New task",
Task.class);
    assertEquals(201, createResponse.getStatusCode().value());
    // ... this test exercises the ENTIRE stack: controller -> service -> repository ->
    real (H2) database
}
}

```

Testing Pyramid Diagram (Full Depth in Book 15)



Internal Working

- **Unit tests needing zero Spring context** is the direct, concrete payoff of Book 11, Ch.5's constructor-injection recommendation — `new TaskBusinessService(mockRepo)` works instantly, in milliseconds, with no application context startup cost at all; field-injected classes cannot be tested this way without significant extra ceremony (reflection or actually starting a test context), which is precisely why constructor injection's testability advantage isn't abstract — it's measured in real test-suite execution speed.
- `@WebMvcTest` loads **only** the web layer (controllers, `@ControllerAdvice`, Jackson configuration) — dramatically faster than a full `@SpringBootTest`, since it doesn't start the database, service beans, or anything outside the HTTP-handling slice; `@MockBean` replaces the real `TaskBusinessService` with a Mockito mock, letting the test focus purely on "does this controller correctly translate HTTP in and out," isolated from business logic correctness (which belongs in Level 1's unit tests).
- The **testing pyramid** shape (many fast unit tests, fewer medium-speed slice tests, very few slow full-integration tests) is a deliberate, standard testing strategy — full depth and justification arrives in Book 15, but this chapter previews *why* Spring Boot's test annotations are deliberately layered to support exactly this pyramid shape, rather than defaulting every test to the slowest, most comprehensive `@SpringBootTest`.

Real-World Example

Telugu: Real CI/CD pipelines (Book 24లో పూర్తిగా) వందల కొద్దీ fast unit tests run చేస్తాయి seconds లో, కొన్ని dozen `@WebMvcTest` medium-speed tests, మరియు కేవలం handful of `@SpringBootTest` slow integration tests — ఈ pyramid shape build time ని manageable గా ఉంచుతుంది, పెద్ద codebases లో కూడా.

English: Real CI/CD pipelines (fully covered in Book 24) run hundreds of fast unit tests in seconds, a few dozen medium-speed `@WebMvcTest` tests, and only a handful of slow `@SpringBootTest`

integration tests — this pyramid shape keeps overall build time manageable even in large codebases, directly motivating why Spring Boot's testing annotations are deliberately layered this way.

Interview Answer

"Spring Boot testing has three layers: pure unit tests (plain JUnit + Mockito, zero Spring context, fastest — the direct payoff of Book 11's constructor-injection recommendation), `@WebMvcTest` (loads only the web layer, uses `MockMvc` and `@MockBean` to test controllers in isolation from business logic), and `@SpringBootTest` (full application context, slowest but most realistic, for true end-to-end integration testing). This layered approach supports the standard testing pyramid — many fast unit tests, fewer slice tests, very few full integration tests — full depth in Book 15."

Cross Questions

- Q: Why can `TaskBusinessServiceUnitTest` instantiate its class under test with plain `new`, without any Spring annotations at all? → A: Because `TaskBusinessService` uses constructor injection (Book 11, Ch.5) — its dependencies are ordinary constructor parameters, so a test can supply a mock directly via `new`, with no need for Spring's container at all.
- Q: What does `@WebMvcTest` load, and what does it NOT load, compared to `@SpringBootTest`? → A: It loads only the web/controller layer (and related MVC infrastructure); it does NOT start the full application context, database, or service-layer beans — `@MockBean` supplies fake versions of dependencies the controller needs.
- Q: Why does the testing pyramid favor many fast unit tests over many slow integration tests? → A: Fast tests give quick feedback and keep CI/CD build times manageable at scale; slow, full-stack integration tests are still valuable for catching real cross-layer issues, but running hundreds of them would make builds impractically slow.

Tricky Questions

- Q: If a class uses field injection (Book 11, Ch.5's discouraged approach), can it still be unit-tested with plain `new` and Mockito mocks? → A: Not directly — since its dependencies are populated via reflection by the Spring container (or `@InjectMocks`-style reflection tricks in tests), a plain `new` leaves those fields `null`; this is a concrete, measurable testability cost of field injection, not just a style preference.
- Q: Does `@MockBean` in a `@WebMvcTest` replace the real bean everywhere in the (limited) loaded context, or just for that specific test class? → A: Just for that test class/context — `@MockBean` registers the mock into that specific test's Spring context, scoped to that test, not affecting other test classes or the real application.

Coding Exercise

L1: Write a pure unit test (no Spring context) for a constructor-injected service, using a Mockito mock dependency. **L2:** Write a `@WebMvcTest` for a controller, using `@MockBean` to fake its service dependency, and `MockMvc` to verify a JSON response. **L3:** Write a `@SpringBootTest` integration test exercising a full create-then-retrieve flow against an in-memory H2 database. **L4 (Interview):**

Explain the three Spring Boot testing layers and when to use each. **L5 (Senior):** Design a testing strategy for a new feature, specifying roughly how many tests belong at each pyramid level and why. **L6 (Mastery):** Explain, from memory, why constructor injection is what makes fast, Spring-context-free unit testing possible, connecting directly back to Book 11, Ch.5.

CHAPTER 12 — Building a Complete CRUD REST API

Goal

This chapter consolidates Chapters 1-11 into one complete, working `Product` CRUD API, demonstrating every concept applied together before the capstone mini project.

Java Code — The Complete Feature

```
// ENTITY (Book 13 preview)
@jakarta.persistence.Entity
class Product {
    @jakarta.persistence.Id @jakarta.persistence.GeneratedValue
    Long id;
    String name;
    double price;
    int stockQuantity;
    Product() {}
    Product(String name, double price, int stockQuantity) {
        this.name = name; this.price = price; this.stockQuantity = stockQuantity;
    }
}

// REPOSITORY (Ch.7)
interface ProductRepository extends
org.springframework.data.jpa.repository.JpaRepository<Product, Long> {
    java.util.List<Product> findByStockQuantityLessThan(int threshold);
}

// DTOs (Ch.4) with VALIDATION (Ch.5)
record CreateProductRequest(
    @jakarta.validation.constraints.NotBlank String name,
    @jakarta.validation.constraints.Positive double price,
    @jakarta.validation.constraints.PositiveOrZero int stockQuantity) {}

record ProductResponse(Long id, String name, double price, int stockQuantity, String
stockStatus) {
    static ProductResponse from(Product p) {
        String status = p.stockQuantity == 0 ? "OUT_OF_STOCK" : p.stockQuantity < 10 ?
"LOW_STOCK" : "IN_STOCK";
        return new ProductResponse(p.id, p.name, p.price, p.stockQuantity, status);
    }
}

// EXCEPTIONS (Ch.6, Book 04 Ch.6)
class ProductNotFoundException extends RuntimeException {
    ProductNotFoundException(Long id) { super("Product not found: " + id); }
}

// SERVICE (Ch.7, Book 10 Ch.9)
@org.springframework.stereotype.Service
class ProductService {
    private final ProductRepository repository;
    ProductService(ProductRepository repository) { this.repository = repository; }

    Product create(CreateProductRequest request) {
        return repository.save(new Product(request.name(), request.price(),
request.stockQuantity()));
    }
    Product getById(Long id) { return repository.findById(id).orElseThrow(() -> new
ProductNotFoundException(id)); }
    java.util.List<Product> getAll() { return repository.findAll(); }
    java.util.List<Product> getLowStock() { return repository.findByStockQuantityLessThan(10);
}

    void delete(Long id) {
        if (!repository.existsById(id)) throw new ProductNotFoundException(id);
        repository.deleteById(id);
    }
}
```

```

    }
}

// CONTROLLER (Ch.3, Ch.4)
@org.springframework.web.bind.annotation.RestController
@org.springframework.web.bind.annotation.RequestMapping("/api/products")
class ProductController {
    private final ProductService service;
    ProductController(ProductService service) { this.service = service; }

    @org.springframework.web.bind.annotation.PostMapping
    org.springframework.http.ResponseEntity<ProductResponse> create(
        @jakarta.validation.Valid @org.springframework.web.bind.annotation.RequestBody
        CreateProductRequest request) {
        Product created = service.create(request);
        return
    org.springframework.http.ResponseEntity.status(201).body(ProductResponse.from(created));
    }

    @org.springframework.web.bind.annotation.GetMapping("/{id}")
    ProductResponse getById(@org.springframework.web.bind.annotation.PathVariable Long id) {
        return ProductResponse.from(service.getById(id)); // exception ->
Ch.6's global handler
    }

    @org.springframework.web.bind.annotation.GetMapping
    java.util.List<ProductResponse> getAll() {
        return service.getAll().stream().map(ProductResponse::from).toList(); // Book 07,
Ch.6-7
    }

    @org.springframework.web.bind.annotation.GetMapping("/low-stock")
    java.util.List<ProductResponse> lowStock() {
        return service.getLowStock().stream().map(ProductResponse::from).toList();
    }

    @org.springframework.web.bind.annotation.DeleteMapping("/{id}")
    org.springframework.http.ResponseEntity<Void>
    delete(@org.springframework.web.bind.annotation.PathVariable Long id) {
        service.delete(id);
        return org.springframework.http.ResponseEntity.noContent().build();
    }
}

```

What This Demonstrates

Every chapter of this book working together: auto-configuration (Ch.1) wires everything; the project structure (Ch.2) organizes it; `@RestController` (Ch.3) exposes REST endpoints; DTOs (Ch.4) separate the API contract from the entity; `@Valid` (Ch.5) validates input; a `@RestControllerAdvice` (Ch.6, not repeated here — reuse Ch.6's) handles `ProductNotFoundException`; `@Service` / `Repository` (Ch.7) implement layered architecture; configuration (Ch.8) externalizes the low-stock threshold if desired; logging (Ch.9) and Actuator (Ch.10) provide observability; and Ch.11's three testing layers verify it all.

CHAPTER 13 — Production Readiness: Packaging & Deployment Basics

Telugu Explanation

Spring Boot applications ఒక **executable "fat" JAR** గా package అవుతాయి — application code + అన్ని dependencies + embedded Tomcat, అన్నీ ఒక్క JAR file లోపల. `java -jar app.jar` తో directly run అవుతుంది, separate application server అవసరం లేదు. **Docker containerization** ఈ model కి natural extension — ఒక్క JAR, ఒక్క container image, ఎక్కడైనా consistent గా run అవుతుంది.

Professional English Explanation

Spring Boot applications package as an executable **"fat" JAR** — application code, all dependencies, and the embedded Tomcat server, all inside one JAR file. `java -jar app.jar` runs it directly, with no separate application server needed. **Docker containerization** is a natural extension of this model — one JAR, one container image, running consistently anywhere.

Build & Docker Configuration

```
<!-- pom.xml - the Spring Boot Maven plugin produces the executable fat JAR -->
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

```
# Dockerfile - a MULTI-STAGE build (smaller final image, no build tools in production image)
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests # produces target/app.jar

FROM eclipse-temurin:21-jre # JRE only for runtime - smaller, no
compiler needed (Book 03, Ch.1)
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Command-Line Interaction

```
# Build the fat JAR
./mvnw clean package

# Run it directly - no application server installation needed
java -jar target/task-manager-1.0.0.jar

# With profile and JVM options (Book 03's heap tuning, Ch.8's profiles)
java -Xmx512m -jar target/task-manager-1.0.0.jar --spring.profiles.active=prod

# Build and run as a Docker container
docker build -t task-manager:1.0.0 .
docker run -p 8080:8080 -e SPRING_PROFILES_ACTIVE=prod -e DATABASE_URL=jdbc:postgresql://...
task-manager:1.0.0
```

Internal Working

- The **fat JAR** contains not just your compiled classes but every dependency's classes too, plus embedded Tomcat's classes — Spring Boot's Maven/Gradle plugin achieves this via a nested-JAR loading mechanism (a custom `ClassLoader`, Book 03, Ch.2) that can load classes from JARs-within-a-JAR, something the standard JVM classpath mechanism doesn't natively support — this is a genuinely clever piece of engineering, not just "zip everything together."
- The **multi-stage Docker build** (build stage with the full JDK + Maven, final stage with only a JRE, Book 03, Ch.1's JDK-vs-JRE distinction) produces a significantly smaller, more secure final image — the production container never contains the compiler, build tool, or source code, only what's needed to *run* the already-compiled application — directly applying Book 03's JDK/JRE distinction to real deployment practice.
- Passing `--spring.profiles.active=prod` as a command-line argument, or `SPRING_PROFILES_ACTIVE=prod` as an environment variable (Docker's `-e` flag) both work — Spring Boot's property-source resolution (Ch.8) treats command-line args, environment variables, and properties files as different sources in a well-defined precedence order, letting the exact same artifact behave correctly across environments purely through how it's launched.

Real-World Example

Telugu: Real production CI/CD pipelines (Book 24) `mvn package` → Docker image build → container registry push → Kubernetes deployment — ఇదే pipeline, code మార్చకుండా dev/staging/prod అన్నింటికీ వాడతారు, environment-specific behavior మొత్తం Ch.8's profiles + environment variables ద్వారా control అవుతుంది. English: Real production CI/CD pipelines (Book 24) follow exactly `mvn package` → Docker image build → container registry push → Kubernetes deployment — the same pipeline, unchanged, used across dev/staging/prod, with all environment-specific behavior controlled entirely through Ch.8's profiles and environment variables.

Interview Answer

"Spring Boot packages as an executable fat JAR — application code, dependencies, and embedded Tomcat, all in one file, runnable via `java -jar` with no separate application server. This uses a custom nested-JAR classloader mechanism. Docker containerization is the natural next step — a

multi-stage build compiles with a full JDK but ships only a JRE in the final image (smaller, more secure), and the same artifact behaves correctly across environments purely through profile/environment-variable configuration at launch time, with no code changes needed."

Cross Questions

- Q: Why can't the JVM's standard classpath mechanism load classes directly from nested JARs (a JAR inside a JAR)? → A: The standard classloading mechanism (Book 03, Ch.2) expects a flat classpath of directories/JARs, not JARs nested inside other JARs — Spring Boot's build plugin uses a custom classloader specifically designed to handle this nested structure.
- Q: Why use a multi-stage Docker build instead of a single-stage build with the full JDK? → A: The final production image is significantly smaller and more secure, since it excludes the compiler, build tool, and source code — only the JRE (Book 03, Ch.1) and the already-compiled JAR are needed to actually run the application.
- Q: How can the same Docker image behave differently in dev vs prod without rebuilding it? → A: Via environment variables passed at container run time (`-e SPRING_PROFILES_ACTIVE=prod`), which Spring Boot's property-source resolution picks up — the artifact itself never changes, only its runtime configuration.

Tricky Questions

- Q: Does `EXPOSE 8080` in the Dockerfile actually publish the port to the host machine? → A: No — `EXPOSE` is documentation/metadata within the Docker image itself; actually publishing the port to the host requires the `-p 8080:8080` flag on `docker run` (or the equivalent in orchestration configuration), a common point of confusion for those new to Docker.
- Q: If `-Xmx512m` (Book 03's heap sizing) is set too low for the application's actual working set, what's the likely symptom? → A: Frequent, possibly excessive garbage collection activity, degraded throughput, or ultimately `OutOfMemoryError` (Book 03, Ch.9) under load — a direct, practical connection between JVM memory tuning and container resource configuration in real deployments.

Coding Exercise

L1: Build a Spring Boot application into an executable fat JAR and run it directly with `java -jar`. **L2:** Write a multi-stage Dockerfile for the application and build/run it as a container. **L3:** Run the same container twice with different `SPRING_PROFILES_ACTIVE` environment variable values and confirm different configuration takes effect. **L4 (Interview):** Explain what a Spring Boot fat JAR contains and why a custom classloader is needed for it. **L5 (Senior):** Design a production deployment pipeline (build → containerize → deploy) for a Spring Boot service, specifying how environment-specific configuration is injected at each stage. **L6 (Mastery):** Explain, from memory, why a multi-stage Docker build produces a smaller, more secure final image, connecting it to Book 03's JDK-vs-JRE distinction.

CHAPTER 14 — Mini Project: Production-Style REST API

Goal

Combine every concept from this entire book into one complete, production-shaped Spring Boot REST API.

Requirements

1. **Full CRUD** (Ch.3-4, Ch.12) for a domain of your choice (extend the `Product` example, or build a new one — e.g., a library book-lending system).
2. **Layered architecture** (Ch.7) with `@RestController` → `@Service` → `JpaRepository` → `@Entity`, using DTOs throughout (never exposing entities directly).
3. **Validation** (Ch.5) on all write endpoints, with meaningful constraint messages.
4. **Global exception handling** (Ch.6) covering at least 3 custom exceptions plus a safe fallback.
5. **Externalized configuration** (Ch.8) via `@ConfigurationProperties`, with separate `dev / prod` profile files.
6. **Structured logging** (Ch.9) at appropriate levels throughout the service layer, including correct exception logging.
7. **Actuator** (Ch.10) enabled with a custom `HealthIndicator` for at least one simulated external dependency.
8. **All three testing layers** (Ch.11): unit tests for the service, a `@WebMvcTest` for the controller, and one `@SpringBootTest` integration test.
9. **Packaged and containerized** (Ch.13): a working multi-stage Dockerfile, runnable with different profiles via environment variables.

Concepts Reinforced

Every chapter in this book, applied together in one complete, realistic, deployable Spring Boot application — the culmination of Books 09-12's progression from raw JDBC through servlets, DI, and finally full production-grade REST API development.

Stretch Goal

Add an AOP logging aspect (Book 11, Ch.8) timing every service-layer method call, and expose the resulting timing data as a custom Actuator metric via Micrometer.

FINAL REVISION NOTES

- **Auto-configuration:** conditionally-activated pre-written `@Configuration` classes (Book 11's mechanics, automated); always yields to explicit user config (`@ConditionalOnMissingBean`).
 - **Starters:** dependency-aggregation artifacts with no code; BOM guarantees version compatibility.
 - **REST controllers:** `@RestController` = `@Controller` + `@ResponseBody` ; `@PathVariable` vs `@RequestParam` ; `ResponseEntity<T>` for explicit status control.
 - **DTOs:** prevent mass assignment, decouple API contract from entity schema; centralize entity-to-DTO mapping.
 - **Validation:** `@Valid` runs before the controller method body; violations → `MethodArgumentNotValidException` → Ch.6's handler.
 - **Global exception handling:** `@RestControllerAdvice` + `@ExceptionHandler` = Book 04, Ch.8's pattern, formalized; most-specific-match dispatch; always include a safe fallback.
 - **Service/Repository:** HTTP-agnostic business logic; `JpaRepository` generates implementations from interfaces, including derived-query methods from method names.
 - **Configuration:** `@ConfigurationProperties` > scattered `@Value` ; profile-specific files layer over base config; never hardcode secrets — use environment variable placeholders.
 - **Logging:** SLF4J facade + Logback; always `log.error(msg, exception)` , never just the message string; package-specific levels for signal-to-noise control.
 - **Actuator:** secure-by-default (opt-in exposure); custom `HealthIndicator` s roll up into overall health; Kubernetes liveness/readiness probes consume this.
 - **Testing pyramid:** unit (fast, no context, constructor-injection payoff) > `@WebMvcTest` (web layer only) > `@SpringBootTest` (full stack, slowest).
 - **Packaging/deployment:** fat JAR via custom nested classloader; multi-stage Docker builds (JDK to build, JRE to run); same artifact, different behavior via profiles/env vars.
-



CHEAT SHEET

```
@SpringBootApplication = @Configuration + @ComponentScan + @EnableAutoConfiguration
Auto-config: @ConditionalOnClass/@ConditionalOnMissingBean - classpath-driven, always yields to
explicit config
Starters: no code, just curated dependency bundles, BOM-guaranteed compatible versions
@RestController: auto-JSON | @GetMapping/@PostMapping/@PutMapping/@PatchMapping/@DeleteMapping
@PathVariable=URL segment | @RequestParam=query string | ResponseEntity<T>=explicit
status+headers+body
DTO != Entity: prevents mass assignment, decouples API from DB schema, centralize mapping in
ONE method
@Valid: validates BEFORE method body runs | violations -> MethodArgumentNotValidException
@RestControllerAdvice + @ExceptionHandler: centralized, most-specific-match dispatch, ALWAYS a
safe Exception.class fallback
@Service (HTTP-agnostic biz logic) -> JpaRepository (interface, Spring generates impl, derived
queries from method names)
@ConfigurationProperties(prefix): type-safe config binding > scattered @Value | application-
{profile}.properties layers over base
Logging: SLF4J facade + Logback | log.error(msg, exception) ALWAYS pass exception object, never
just .getMessage()
Actuator: /actuator/health,/info exposed by DEFAULT, others opt-in | custom HealthIndicator
rolls into overall status
Testing pyramid: unit(fast,no context,constructor-injection payoff) > @WebMvcTest(web
layer+@MockBean) > @SpringBootTest(full stack)
Packaging: fat JAR (nested classloader) | multi-stage Docker (JDK build -> JRE run) | same
artifact, config via profiles/env vars
```



INTERVIEW QUESTION BANK — Spring Boot

Beginner

1. What does `@SpringBootApplication` do?
2. What is a Spring Boot starter?
3. What is the difference between `@Controller` and `@RestController`?
4. What does `@Valid` do?
5. What is Spring Boot Actuator used for?

Intermediate 6. Explain how auto-configuration decides whether to activate a given configuration. 7. Why should you use DTOs instead of exposing entities directly in API responses? 8. Explain how `@RestControllerAdvice` centralizes exception handling. 9. What is the difference between `@WebMvcTest` and `@SpringBootTest`? 10. Why is `log.error(msg, exception)` important, and what's lost without it?

Advanced 11. Explain the fat JAR packaging mechanism and why a custom classloader is needed. 12. Explain mass assignment and how request DTOs prevent it. 13. Explain how a `JpaRepository` interface gets a working implementation with no code written by the developer. 14. Why does constructor injection (Book 11) directly enable fast, Spring-context-free unit testing? 15. Explain why Actuator doesn't expose all endpoints by default, and the risk of exposing `/actuator/env` unauthenticated.

Senior/Architect 16. Design a complete layered, validated, exception-handled REST API for a new domain, explaining every architectural choice. 17. Design a multi-environment (dev/staging/prod) configuration and deployment strategy using profiles, externalized config, and Docker. 18. Explain, end-to-end, what happens from `java -jar app.jar` to a successfully handled REST request. 19. Review an API directly exposing JPA entities as REST responses — explain the risks and the DTO-based remediation. 20. Design a production observability strategy combining Actuator health checks, custom metrics, and structured logging for a new service.

(Full short/professional/deep-senior answer scaffolding for every question across the series lives in Book 23 — Java Interview Master Book.)



CROSS-QUESTION ENGINE — Sample Chains

Q: What is Spring Boot auto-configuration? → Q: How does it decide what to configure? → Q: What happens if you define your own bean of a type it would auto-configure? → Q: Is this the same IoC container as plain Spring, or something different? → Q: Name 3 things auto-configured when spring-boot-starter-web is on the classpath.

Q: Why use DTOs instead of returning entities directly? → Q: What is mass assignment? → Q: How does a request DTO prevent it? → Q: What's the risk of using the SAME class for both request and response? → Q: Where should entity-to-DTO mapping logic live?

Q: How does global exception handling work in Spring Boot? → Q: What annotation enables it? → Q: How does Spring pick which @ExceptionHandler to use? → Q: Why is a fallback Exception.class handler necessary? → Q: How does this relate to what you built manually in Book 04?



CONSOLIDATED EXERCISES (All Levels)

L1 — Beginner: Redo each chapter's L1 exercise unaided. **L2 — Intermediate:** Redo each chapter's L2/L3 exercises, explaining every Spring Boot mechanic out loud in Telugu. **L3 — Advanced:** Build a complete CRUD REST API for a new domain, using every technique from Ch.1-9. **L4 — Interview:** Answer all 20 Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Chapter 14 mini project fully, including the AOP/Actuator metric stretch goal. **L6 — Mastery:** Teach Chapters 4 (DTOs), 6 (global exception handling), and 11 (testing layers) out loud, from memory, using fresh examples.



ONE-DAY REVISION PLAN (≈6.5 hours)

Time	Focus
0:00–0:30	Ch.1–2: Auto-configuration, project structure/starters
0:30–1:00	Ch.3–4: REST controllers, DTOs — the mass-assignment security point
1:00–1:30	Ch.5: Validation
1:30–1:45	Break
1:45–2:30	Ch.6: Global exception handling — the highest-density interview block
2:30–3:00	Ch.7: Service/Repository layers
3:00–3:30	Ch.8: Configuration/profiles
3:30–4:00	Ch.9: Logging — the log.error(msg,e) gotcha
4:00–4:30	Ch.10: Actuator
4:30–5:15	Ch.11: Testing layers — the constructor-injection testability payoff
5:15–5:45	Ch.12: Complete CRUD walkthrough
5:45–6:15	Ch.13: Packaging/deployment
6:15–6:30	Interview Bank — answer all questions from memory



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1-3 (auto-config, structure, controllers) — build and run a minimal Spring Boot app
2	Ch.4-6 (DTOs, validation, global exception handling) — build a fully validated, exception-safe endpoint
3	Ch.7-8 (service/repository, configuration) — build the layered structure with profiles
4	Ch.9-10 (logging, Actuator) — instrument the app with proper logging and a custom health indicator
5	Ch.11 (testing) — write all 3 test layers for one feature
6	Ch.12-13 + Mini Project — build the complete CRUD API and containerize it
7	Full mock interview: all 20 bank questions + all cross-question chains, in Telugu and English, without notes



FINAL MASTERY CHECKLIST

- ☐ I can explain what @SpringBootApplication expands to and how auto-configuration decides what to activate.
- ☐ I can build a full REST controller using the @RequestMapping family with correct status codes.
- ☐ I can design request/response DTOs that prevent mass assignment.
- ☐ I can apply Bean Validation and explain when it runs relative to the controller method.
- ☐ I can build a @RestControllerAdvice with a safe fallback handler.
- ☐ I can structure a layered Service/Repository architecture using Spring Data JPA.
- ☐ I can externalize configuration via @ConfigurationProperties and profiles.
- ☐ I can apply correct structured logging, including proper exception logging.
- ☐ I can configure Actuator securely and write a custom HealthIndicator.
- ☐ I can write tests at all three Spring Boot testing layers.
- ☐ I can package and containerize a Spring Boot application with multi-environment configuration.
- ☐ I built the Production-Style REST API mini project, including the AOP/metrics stretch goal.
- ☐ I answered the full Interview Question Bank confidently in both Telugu and English.

Once every box is checked, you are ready for `13_Spring_Data_JPA_Hibernate.md`.